

Flight Price Prediction — Complete Walkthrough + Interview Prep (200 Q&A)

Ye document tera pura project **start se end tak line-by-line** samjhata hai (kya kiya, kaise kiya, kyun kiya), aur last mein **200 interview questions** hain (Statistics, ML, Feature Engineering, XGBoost, AWS, Deployment) — sab tere hi project se related, taaki interview mein koi bhi "minute detail" pooche to tu confidently jawab de sake.

PART 0 — Project ka Bird's Eye View

Problem: Indian domestic flights ka price predict karna hai — Airline, Source, Destination, Date, Time, Duration, Stops, Additional Info jaise features dekh kar.

Pipeline (end-to-end flow):

```
Raw CSV (flight_price.csv, 10,683 rows)
↓ [DataCleaning.ipynb]
Cleaned data (10,462 rows) → train/val/test split
↓ [EDA.ipynb]
Statistical understanding + hypothesis testing
↓ [FeatueEngineering.ipynb → baseline preprocessing pipeline]
↓ [TrainingData.ipynb → improved pipeline + AWS SageMaker training]
Preprocessed data → S3 → SageMaker XGBoost (Bayesian tuning, 20 jobs)
↓
Best model → xgboost-model.json (R2 Test = 0.79)
↓ [convert.py → pickle to JSON conversion]
↓ [app.py → Streamlit web app]
User Input → Live Prediction (₹)
```

Tech stack: Python 3.11, pandas, numpy, scikit-learn, feature-engine, XGBoost 1.2-1, AWS SageMaker + S3, Streamlit.

Files/notebooks jo humne dekhe (order of execution): 1. `DataCleaning.ipynb` (root, aur same copy `Notebook/DataCleaning.ipynb`) — raw data clean karke train/val/test bana ke save karta hai. 2. `Notebook/EDA.ipynb` — cleaned train data ka deep statistical + visual analysis. 3. `Notebook/FeatueEngineering.ipynb` — baseline sklearn preprocessing pipeline prototype karta hai (local machine par). 4. `Notebook/TrainingData.ipynb` — usi pipeline ko **improve** karke (5 changes), AWS SageMaker par XGBoost train + tune karta hai. 5. `app.py` — Streamlit app; same improved pipeline ko dobara define karke, live prediction deta hai. 6. `convert.py` — SageMaker se aaya pickled model ko JSON format mein convert karta hai.

Chalo ab har file ko **line by line** samjhte hain.

PART 1 — `DataCleaning.ipynb` (Data ko Saaf Karna)

Step 1: Imports aur Setup

```
import os, numpy as np, pandas as pd
from sklearn.model_selection import train_test_split
```

- `os` → file paths banane ke liye (Windows path `PROJECT_DIR` set kiya hai).
- `train_test_split` → baad mein data ko train/val/test mein todne ke liye.

Ek helper function `get_data(name)` banaya jo `Data/{name}.csv` read karta hai — reusable function taaki baar baar path na likhna pade.

Step 2: Raw Data Padhna

`flights = get_data("flight_price")` → **10,683 rows, 11 columns**: `Airline, Date_of_Journey, Source, Destination, Route, Dep_Time, Arrival_Time, Duration, Total_Stops, Additional_Info, Price`

`flights.info()` se pata chala: - Sab columns `object` (string) type hain, sirf `Price` `int64` hai — **yeh problem hai** kyunki Date, Time, Duration, Stops sab conceptually numeric/datetime hain but string ke roop mein stored hain. - `Route` aur `Total_Stops` mein 1-1 missing value hai.

Kyun important hai: Kisi bhi ML model ko train karne se pehle data types sahi karna zaroori hai — string "22:20" ko model directly samajh nahi sakta, usse time object ya minutes mein convert karna padega.

Step 3: Duplicates Check

```
flights.duplicated().sum()    # → 220
```

220 fully-duplicate rows mile (same `Airline+Date+Source+...` sab same). Ye galti se data scraping/collection ke time repeat ho gaye honge. In cell 15 mein `.loc[flights.duplicated(keep=False)]` se saare duplicate pairs dekhe gaye (jaise ek hi flight do baar list ho gayi).

Decision: Baad mein `drop_duplicates()` se hata denge — kyunki duplicate rows model ko galat weight de sakte hain (jaise ek hi flight price pattern ko 2x importance mil jaye, jo overfitting/bias create karta hai).

Step 4: Column-by-Column Detailed Analysis

Airline:

```
flights.Airline.unique()
```

12 unique values mile, lekin kuch inconsistent hain: `'Vistara Premium economy'`, `'Jet Airways Business'`, `'Multiple carriers Premium economy'` — ye actually same airline ke sub-categories hain jo unnecessary noise create kar rahe. Fix: `" Premium economy"` aur `" Business"` suffix hata ke, `.str.title()` se consistent casing.

Date_of_Journey: String format `"24/03/2019"` (DD/MM/YYYY). `pd.to_datetime(..., dayfirst=True)` se proper datetime banaya — `dayfirst=True` isliye zaroori hai kyunki Indian format mein din pehle aata hai, warna pandas `03/04` ko March-4 na maan ke April-3 maan lega (US format confusion).

Source / Destination: Simple categorical, `.unique()` check kiya — 5 sources, 6 destinations, koi issue nahi.

Dep_Time / Arrival_Time: Interesting catch — `Arrival_Time` mein kuch values `"01:10 22 Mar"` jaise hain (yani agle din pahunchi flight, isme date bhi included hai). Sirf time nikalna hai isliye `pd.to_datetime(...).dt.time` use kiya — jo extra date part ko automatically ignore kar deta hai aur sirf HH:MM:SS return karta hai.

Duration: String format `"2h 50m"` ya sirf `"19h"` ya sirf `"5m"` (koi bhi part missing ho sakta hai). Ek custom function `convert_to_minutes()` likha:

```
def convert_to_minutes(ser):
    return (ser.str.split(" ", expand=True)
            .set_axis(["hour", "minute"], axis=1)
            .assign(hour=lambda d: d.hour.str.replace("h", "").astype(int).mul(60),
                    minute=lambda d: d.minute.str.replace("m", "").fillna("0").astype(int))
            .sum(axis=1))
```

Kaise kaam karta hai: String ko space se split karke hour/minute columns banaye, "h"/"m" letters hata ke int banaya, hour ko 60 se multiply karke total minutes mein convert kiya, dono add kar diya. `fillna("0")` isliye taaki jab sirf "19h" ho (minute missing) to error na aaye.

Data quality catch (index 6474): Ek row mili jisme Duration sirf `"5m"` hai (koi "h" nahi) — matlab flight sirf 5 minute ki?! Ye clearly ek **data entry error** hai (real flight 5 min mein complete nahi ho sakti), isliye is row ko explicitly `.drop(index=6474)` se hata diya. **Yeh domain knowledge se outlier pakadne ka classic example hai** — sirf statistics (IQR/z-score) se ye catch nahi hota kyunki 5 min bhi ek "valid looking number" hai, lekin business context se pata chalta hai ye galat hai.

Total_Stops: `"non-stop"`, `"1 stop"`, `"2 stops"` jaise strings the. `"non-stop"` ko `"0"` se replace kiya, phir regex `" stops?"` (yaani "stop" ya "stops" dono match) hata ke numeric bana diya.

Additional_Info: `"No info"` aur `"No Info"` dono forms mil rahi thi (inconsistent casing) — isko ek jaisa `"No Info"` bana diya.

Step 5: `clean_data()` — Sab kuch ek function mein combine

Ye ek master function hai jo upar ke saare steps ko method-chaining (pandas `.pipe/.assign` style) mein combine karta hai: 1. Bad row (6474) drop 2. Duplicates drop 3. Saare text columns ke whitespace `.strip()` (agar koi extra spaces hon) 4. Column names lowercase (`airline`, `date_of_journey`, etc.) 5. Airline naam clean 6. Date/time/duration/stops sab convert 7. `route` column pura drop kar diya (is version mein route use nahi kiya — sirf source+destination use ho raha hai, jo simpler hai lekin thoda information loss hai)

Result: `flights_cleaned` → **10,462 rows × 10 columns** (11 se ek column `route` kam ho gaya).

Step 6: Train / Val / Test Split

```
X = flights_cleaned.drop(columns="price")
y = flights_cleaned.price.copy()

X_, X_test, y_, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
X_train, X_val, y_train, y_val = train_test_split(X_, y_, test_size=0.2, random_state=42)
```

Do-step split kyun? Sklearn ka `train_test_split` sirf 2-way split karta hai, isliye 3-way (train/val/test) ke liye **do baar** call karna padta hai: - Pehle 20% test ke liye alag kiya (yeh puri training process se completely untouched rahega, sirf final evaluation ke liye). - Bache hue 80% mein se phir 20% val ke liye nikala (yeh hyperparameter tuning ke dauraan use hoga).

Final proportions: Train ≈ 64%, Val ≈ 16%, Test ≈ 20% → counts: **Train = 6,695 | Val = 1,674 | Test = 2,093.**

`random_state=42` — reproducibility ke liye (har baar run karne pe same split mile, taaki results compare kar sako).

Step 7: Export

`export_data()` function teeno subsets ko `train.csv`, `val.csv`, `test.csv` mein save karta hai (X aur y ko `.join()` karke, taaki price column bhi saath mein rahe).

PART 2 — Notebook/EDA.ipynb (Exploratory Data Analysis)

Yahan se ek naya helper module import hota hai: `eda_helper_functions.py` — isme saare reusable plotting/stat-testing functions hain (isko bhi hum niche detail mein dekhenge).

Step 1: Data Load + Type Fix

`train.csv` (6,695 rows) padha, phir date/time columns ko wapas `pd.to_datetime(..., format="mixed")` se datetime bana liya (CSV mein save/load karne se datetime wapas string ban jaata hai, isliye re-parse zaroori hai).

Step 2: High-Level Summary

`train.describe(include="number")` → duration, total_stops, price ka mean/std/min/max/quartiles.
`train.describe(include="O")` → categorical summary (mode, unique counts) — total_stops ko temporarily object bana ke describe kiya taaki uski category-jaisi nature dikh sake.

Step 3: Missing Values

Custom function `missing_info()` banaya jo har column ka missing count + percentage return karta hai (bar chart bhi bana sakta hai `plot_missing_info()`). **Finding:** sirf `total_stops` mein 1 missing value. Us row ko dekha, aur **domain assumption** lagayi: missing stop info matlab likely non-stop flight hoga (chunki non-stop sabse common category hai), isliye `.fillna(0)` kar diya.

Step 4: Outlier Detection — Isolation Forest

```
forest = IsolationForest(n_estimators=10, random_state=42)
train.assign(outlier=forest.fit_predict(train.drop(columns="price").select_dtypes(include="number")))
.query("outlier == -1")
```

Isolation Forest kya karta hai: Ye ek **unsupervised anomaly detection** algorithm hai — random decision trees banata hai jo data ko randomly split karte hain; jo points **kam splits mein hi isolate ho jaate hain** (matlab baaki data se bahut alag hain), unko anomaly/outlier maana jaata hai (`-1` label). Yahan sirf numeric features (duration, total_stops) par lagaya gaya — pure inspection/insight ke liye, is notebook mein directly rows delete nahi ki gayi (actual outlier handling baad mein Feature Engineering step mein Winsorizer se hota hai).

Step 5: Correlation Analysis

- `pair_plots()` — saare numeric columns ka scatterplot matrix (lower triangle only, taaki redundant upper-triangle na dikhe).
- `correlation_heatmap()` — **Spearman correlation** use kiya (Pearson nahi!). Spearman rank-based hota hai, isliye non-linear monotonic relationships aur outliers ko Pearson se better handle karta hai — jo yahan zaroori tha kyunki duration/price dono skewed hain.
- `cramersV_heatmap()` — categorical columns (airline, source, destination, additional_info) ke beech association strength dekhne ke liye **Cramér's V** use kiya (ye Chi-square test par based hota hai, but 0 se 1 ke beech normalized value deta hai jo interpret karna easy hota hai — jaise correlation coefficient but categorical variables ke liye).

Step 6: Feature-by-Feature Deep Dive (sabse important part)

Har feature ke liye ek consistent **3-part ritual** follow kiya gaya: 1. **Summary/univariate plots** (bar chart, pie chart, word cloud, histogram, box plot, QQ plot) 2. **Bivariate plots** (feature vs price — bar/box/violin plots) 3. **Hypothesis Testing** (statistically prove karna ki association real hai ya random)

- **Airline vs Price:** - `num_cat_hyp_testing()` do tests karta hai: - **ANOVA (F-test):** Null Hypothesis = "sab airlines ka mean price same hai". Result: p-value ≈ 0.0 (statistic=473.28) → **Reject null** → airlines ka price alag-alag hai. - **Kruskal-Wallis test:** ANOVA ka non-parametric version (medians compare karta hai, normality assume nahi karta) — usi conclusion pe pahuncha ($p \approx 0$). - **Dono tests kyun?** ANOVA normality assume karta hai (jo price data mein nahi hai — price right-skewed hai), isliye Kruskal-Wallis ek **robustness check** ke roop mein bhi chalaya gaya, taaki result sirf normality-violation ki wajah se galat na ho.

- **Date_of_Journey vs Price:** Monthly aur weekly average price ke bar plots banaye (`pd.Grouper(freq="M") / freq="W"`) — pattern dekha ki kaunse mahine mein prices zyada hain (seasonality check).

- **Dep_Time vs Price:** Time ko `morning/afternoon/evening/night` buckets mein convert karke (via `np.select`) price ke saath bivariate analysis kiya.

- **Duration — sabse detailed analysis (`num_summary()`):** - **Percentiles table:** min=75, 25%=170, median=510, 75%=922.5, max=2860 minutes. - **Central tendency:** mean(633.5) vs trimmed mean vs median(510) — **mean > median** already batata hai ki data **right-skewed** hai (kuch bahut lambi duration wali flights mean ko upar khinch rahi hain). - **Skewness = 0.89** (positive/right skew), **Kurtosis = -0.05** (near-normal tails, thoda platykurtic). - **Normality Tests:** - **Shapiro-Wilk test:** H_0 = "data normally distributed". p-value = $7.4e-57$ (extremely small) → **Reject H_0** → data normal nahi hai. - **Anderson-Darling test:** Test statistic (248.46) >> critical value (0.787) → yahan bhi **Reject H_0** . - **Dono tests kyun:** Shapiro-Wilk chhote-medium samples ke liye powerful hota hai, Anderson-Darling tails ko zyada weight deta hai — dono se confirm hota hai duration Gaussian nahi hai, isliye baad mein PowerTransformer/RBF jaisi non-linear techniques use karna justified hai. - `num_num_hyp_testing()` — **Duration vs Price association:** - **Pearson $r = 0.509$** (linear correlation) - **Spearman $\rho = 0.698$** (monotonic/rank correlation) - Dono ka p-value ≈ 0 → statistically significant correlation hai. - **Important insight:** Spearman (0.698) > Pearson (0.509) — iska matlab relationship **linear nahi, balki monotonic non-linear** hai. Yehi cheez baad mein Feature Engineering mein **RBF kernel similarity** use karne ka justification banti hai (linear scaling kaafi nahi hoga). - **IQR-based outlier check:** Lower=-958.75, Upper=2051.25 minutes → jo flights 2051 min (~34 hrs) se zyada lambi hain unhe outlier maana — ye baad mein `Winsorizer` se cap kiye jaayenge.

- **Additional_Info:** `cat_summary()` se dikha ki 7 categories hain lekin **severe imbalance:** "No Info" = 78.4%, "In-flight meal not included" = 18.1%, baaki 5 categories combined <4%. Itni rare categories ko as-is one-hot encode karna noise create karega, isliye decision liya gaya ki isko baad mein binary flag (`have_info`) mein bhi convert karenge.

Step 7: Automated EDA Report

```
from ydata_profiling import ProfileReport
report = ProfileReport(train)
report.to_file(output_file="output.html")
```

`ydata_profiling` (pehle `pandas-profiling` kehata tha) — ek library jo **automatically** poora EDA report (distributions, correlations, missing values, warnings) ek HTML file mein generate kar deti hai. Ye manual EDA ka **cross-check/supplement** hai — jaldi se overall data quality verify karne ke liye.

PART 3 — `Notebook/FeatueEngineering.ipynb` (Baseline Preprocessing Pipeline)

Ye notebook **local machine** par pipeline **prototype/design** karne ke liye hai — is pipeline ka improved version hi baad mein SageMaker training aur Streamlit app mein use hota hai. Isliye is part ko dhyan se samjhna zaroori hai — yahi core engineering hai.

Setup

```
sklearn.set_config(transform_output="pandas")
```

Ye global setting sklearn ke transformers ko force karti hai ki wo **numpy array ki jagah pandas DataFrame** return karein (named columns ke saath). Isse debugging easy ho jaati hai — tu directly dekh sakta hai ki kaunse column se kya feature bana.

`X_train = train.drop(columns="price")`, `y_train = train.price` — features aur target alag kiye.

Ab har original column ke liye ek **chhota sa mini-pipeline** banaya gaya hai, aur end mein sabko `ColumnTransformer` se jodenge.

4.1 — `airline` column

```
air_transformer = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("grouper", RareLabelEncoder(tol=0.1, replace_with="Other", n_categories=2)),
    ("encoder", OneHotEncoder(sparse_output=False, handle_unknown="ignore"))
])
```

- **SimpleImputer(most_frequent)** — agar koi missing airline ho to sabse common airline se fill (safety net, is dataset mein actually koi missing nahi hai but production-ready coding practice hai).
- **RareLabelEncoder(tol=0.1)** — jo bhi category dataset ka **10% se kam** hissa hai, use `"Other"` mein group kar deta hai. `n_categories=2` guarantee karta hai ki kam se kam top-2 categories hamesha alag rakhi jaayen (chahe wo bhi 10% se kam ho). **Kyun zaroori hai:** Agar Trujet jaisi airline sirf 1 baar aayi hai to usko alag one-hot column dena waste hai — ye "curse of dimensionality" aur overfitting-on-rare-category dono se bachata hai.
- **OneHotEncoder(handle_unknown="ignore")** — categories ko binary dummy columns mein convert karta hai. `handle_unknown="ignore"` **production ke liye critical** hai — agar live app mein koi naya/anjaan airline aa jaaye jo training data mein nahi tha, to error crash karne ke bajaye wo row sab-zero encode ho jaayegi (graceful handling).

- **Result:** 9 raw airlines → 5 columns (Air India, Indigo, Jet Airways, Multiple Carriers, Other).

4.2 — date_of_journey

```
doj_transformer = Pipeline([
    ("dt", DatetimeFeatures(features_to_extract=["month", "week", "day_of_week", "day_of_year"], yearfirst=True)),
    ("scaler", MinMaxScaler())
])
```

Ek single date column se **4 naye numeric features** nikale (`feature_engine` library ka `DatetimeFeatures` transformer): month, week-of-year, day-of-week, day-of-year. Phir `MinMaxScaler` se 0-1 range mein scale kiya.

Kyun date ko todna zaroori hai: Raw date (jaise "2019-03-21") ek model ke liye directly meaningless number hoti hai. Lekin "month" seasonality capture karta hai (summer vacations mein prices zyada), "day_of_week" weekday/weekend pattern capture karta hai, "day_of_year" overall trend capture karta hai.

MinMaxScaler kyun (StandardScaler nahi)? Tree-based models (XGBoost) ke liye scaling technically zaroori nahi hoti (trees sirf split-points dhoondte hain, scale-invariant hote hain), lekin consistency aur agar kabhi doosra model (linear/distance-based) use karna pade to pipeline reusable rahe, isliye best-practice ke taur pe scaling maintain ki gayi.

4.3 — source & destination

```
location_transformer = Pipeline([
    ("grouper", RareLabelEncoder(tol=0.1, replace_with="Other", n_categories=2)),
    ("encoder", MeanEncoder()),
    ("scaler", PowerTransformer())
])
```

- **MeanEncoder (Target/Mean Encoding):** Har city ko uske corresponding **average price** se replace kar deta hai (jaise Bangalore → Bangalore se jaane wali sab flights ka mean price). Ye high-cardinality categorical variables ke liye **one-hot se zyada powerful** hai kyunki ye directly target ke saath relationship encode karta hai.
- **Note: Risk (interview mein pooche jaane wala point):** Mean encoding mein **data leakage** ka risk hota hai agar training data pe hi fit-transform kiya jaaye without cross-validation folding — kyunki row ka apna price hi uske group-mean mein contribute kar raha hota hai. Yahan ye risk isliye manageable hai kyunki encoder sirf `X_train` pe `fit` hota hai aur baad mein val/test/naya data sirf `.transform()` hota hai (fit dobara nahi hota) — lekin train set ke andar hi thoda leakage/overfitting rehta hai (`feature_engine` ka `MeanEncoder` by default simple leave-in encoding karta hai, cross-fold nahi).
- **PowerTransformer** — mean-encoded values ko zyada Gaussian-jaisi shape deta hai (Yeo-Johnson method by default, jo negative/zero values bhi handle kar sakta hai) — helps outlier/skew ko soften karne mein.

4.4 — dep_time & arrival_time

Do parallel pipelines, `FeatureUnion` se combine: - **time_pipe1:** `DatetimeFeatures` se hour+minute nikale, `MinMaxScaler`. - **time_pipe2:** Custom function `part_of_day()` — `np.select()` se hour ko 4 buckets mein daala: morning(4-12), afternoon(12-16), evening(16-20), night(baaki sab). Phir `CountFrequencyEncoder` (category ko uske dataset mein occurrence-frequency se replace karta hai — arbitrary numeric-order assume nahi karta jaisa `OrdinalEncoder` karta), phir `MinMaxScaler`.

FeatureUnion kya hai: `ColumnTransformer` alag-alag **columns** ko alag transformer deta hai, jabki **FeatureUnion same column(s) ko multiple alag-alag transformers se guzarta hai aur results ko side-by-side jod deta hai** — yahan same `dep_time/arrival_time` se do alag "views" (raw hour/minute + part-of-day pattern) nikaale ja rahe hain.

4.5 — **duration** (sabse complex feature engineering yahan hai)

Custom Transformer likha gaya — `RBFPercentileSimilarity`:

```
class RBFPercentileSimilarity(BaseEstimator, TransformerMixin):
    def __init__(self, variables=None, percentiles=[0.25, 0.5, 0.75], gamma=0.1):
        ...
    def fit(self, X, y=None):
        self.reference_values_ = {col: X[col].quantile(self.percentiles)... for col in self.variables}
        return self
    def transform(self, X):
        # rbf_kernel(X, reference_points, gamma) — Gaussian similarity
```

Ye kya karta hai (bahut important interview topic): - `fit()` mein training data ke 25th/50th/75th percentile duration values ko "reference anchor points" bana leta hai (baseline mein 3 anchors: [170, 510, 922.5] minutes). - `transform()` mein har duration value ka **RBF (Gaussian) kernel similarity score** nikalta hai un anchors ke saath: $\text{similarity} = \exp(-\text{gamma} * (x - \text{anchor})^2)$. - **Matlab:** Agar ek flight ki duration 510 min (median ke bahut kareeb) hai, to `duration_rbf_50` ≈ 1 (very similar), aur `duration_rbf_25/duration_rbf_75` kam honge. - **Kyun ye technique use ki (linear scaling ki jagah)?** EDA mein humne dekha tha ki duration-price relationship **non-linear/monotonic** hai (Spearman 0.698 >> Pearson 0.509). RBF similarity model ko batati hai "ye flight kis typical duration-bucket ke sabse kareeb hai" — ye soft, smooth, non-linear signal deta hai jo hard binning (jaise sirf short/medium/long categories) se better hai kyunki boundaries pe abrupt jump nahi hota. - `gamma=0.1` — controls kernel ki "width": chhota gamma = broader/smoother similarity (dur values door tak bhi similar maane jaate), bada gamma = sirf bahut close values hi similar maani jaati.

Baaki duration sub-features (FeatureUnion se 4 parallel views): 1. `RBFPercentileSimilarity` → `PowerTransformer` — non-linear similarity signal. 2. `duration_category()` (custom function: short<180min, medium 180-400min, long≥400min) → `OrdinalEncoder(categories=[["short", "medium", "long"]])` — explicit order diya kyunki ye ek natural ordinal ranking hai. 3. `is_over(1000)` — binary flag agar duration ≥1000 min (~16.7 hrs) — extreme long-haul/multi-stop flights ko highlight karta hai. 4. `StandardScaler` — raw duration ko z-score normalize karke bhi rakha (taaki raw continuous signal bhi available rahe, sirf transformed versions hi nahi).

Pehle outlier handling + imputation:

```
duration_transformer = Pipeline([
    ("outliers", Winsorizer(capping_method="iqr", fold=1.5)),
    ("imputer", SimpleImputer(strategy="median")),
    ("union", duration_union)
])
```

- **Winsorizer(iqr, fold=1.5)** — EDA mein jo IQR outlier bounds nikale the (upper≈2051 min), unse zyada extreme values ko **delete nahi karta balki us boundary value pe cap kar deta hai** ("clip" karna). Ye rows delete karne se better hai kyunki sample size preserve rehta hai, sirf extreme influence kam hoti hai.
- **SimpleImputer(median)** — agar koi missing duration ho to median se fill (mean nahi, kyunki data skewed hai aur median outliers se robust hota hai).

Yeh "multi-view" engineering ka concept samajhna zaroori hai: Ek hi raw column (duration) se 4 different mathematical "lenses" banaye — raw scaled value + non-linear similarity + coarse category + extreme-flag. Tree-based model (XGBoost) ko multiple split-options milte hain, jisse woh better patterns dhoondh sakta hai.

4.6 — `total_stops`

```
def is_direct(X): return X.assign(is_direct_flight=X.total_stops.eq(0).astype(int))
total_stops_transformer = Pipeline([("imputer", SimpleImputer("most_frequent")), ("", FunctionTransformer
```

Missing value ko most-frequent (0 stops) se fill, aur ek naya binary flag `is_direct_flight` banaya — domain insight ye hai ki direct/non-stop flights ka pricing pattern alag hota hai (convenience premium ya kabhi kam distance ki wajah se sasti bhi).

4.7 — `additional_info`

Do parallel views (`FeatureUnion`): 1. `RareLabelEncoder` → `OneHotEncoder` — rare categories (jinka freq <10%) ko "Other" mein group karke one-hot (result: 3 columns — No Info / In-flight meal not included / Other). 2. `have_info()` custom binary flag — 1 agar `additional_info != "No Info"` (yaani "kya is flight ke saath koi special note hai ya nahi", chahe wo note kuch bhi ho) — ye coarser lekin robust signal hai jab individual categories bahut rare hon.

4.8 — Sabko Jodna: `ColumnTransformer`

```
column_transformer = ColumnTransformer([
    ("air", air_transformer, ["airline"]),
    ("doj", doj_transformer, ["date_of_journey"]),
    ("location", location_transformer, ["source", "destination"]),
    ("time", time_transformer, ["dep_time", "arrival_time"]),
    ("dur", duration_transformer, ["duration"]),
    ("stops", total_stops_transformer, ["total_stops"]),
    ("info", info_transformer, ["additional_info"])
], remainder="passthrough")
```

Ye har original column-group ko uske respective mini-pipeline se guzaarta hai, aur sab outputs ko side-by-side (horizontally) concatenate kar deta hai. `remainder="passthrough"` ek safety default hai — agar koi column list mein explicitly mention nahi hui to use as-is (bina transform kiye) aage bhej dega (yahan actually sab 9 columns already covered hain, koi extra nahi bacha).

Result: 9 raw columns → **31 engineered columns**.

Step 5: Feature Selection

```
estimator = RandomForestRegressor(n_estimators=10, max_depth=3, random_state=42)
selector = SelectBySingleFeaturePerformance(estimator=estimator, scoring="r2", threshold=0.1)
```

Ye kya karta hai: Har ek engineered feature (31 mein se) ko **akela** (individually, ek time pe ek feature) leke ek chhota RandomForest model train karta hai price predict karne ke liye, aur uska standalone R^2 score note karta hai. Jo features standalone **threshold** (yahan 0.1) se **kam** R^2 dete hain, unhe **drop** kar diya jaata hai.

Example scores (actual output se): - `dur_duration` = 0.425 [OK] (sabse strong akela predictor) - `stops_total_stops` = 0.403 [OK] - `dur_duration_cat` = 0.395 [OK] - `stops_is_direct_flight` = 0.379 [OK] - `air_airline_Air India` = 0.0017 [X] (drop) - `time_dep_time_part_of_day` = -0.0013 [X]

(drop, negative bhi ho sakta hai R^2 agar model random se bhi bura predict kare)

Baseline result: 31 → 13 features selected (threshold=0.1).

Note: Important limitation (interview mein zaroor poocha jaayega): Ye ek **univariate/greedy filter method** hai — ye har feature ko akela test karta hai, **feature-interactions ko ignore karta hai**. Ho sakta hai koi feature akele weak ho lekin doosre feature ke combination mein bahut useful ho — aisi feature yahan galti se drop ho sakti hai. Iska trade-off hai: **simplicity aur speed** vs **completeness**.

Final `preprocessor.pkl` mein save kiya (`pickle.dump`).

PART 4 — `Notebook/TrainingData.ipynb` (AWS SageMaker Training — Production Pipeline)

Ye notebook AWS SageMaker (cloud) mein chalta hai. Isme upar wali FeatureEngineering pipeline ko **5 improvements** ke saath repeat kiya gaya hai, phir usse actual XGBoost model train + tune kiya gaya hai.

5 Improvements (Baseline → Improved)

#	Feature	Baseline	Improved	Kyun
1	Location	Sirf MeanEncoder + PowerTransformer	+ <code>is_north</code> binary flag	North India (Delhi/Kolkata/Mumbai/New Delhi) vs baaki cities ka pricing pattern alag ho sakta hai — domain-based geography feature
2	Time	hour/min + part-of-day	+ <code>is_peak_hour</code> flag (6-9AM ya 5-8PM)	Peak commute hours mein demand zyada = price zyada, ye hypothesis test karne ke liye
3	Duration	3 RBF anchors (25/50/75 percentile)	5 RBF anchors (10/25/50/75/90 percentile)	Zyada anchors = tail behaviour (bahut short ya bahut lambi flights) ko better capture karta hai

4	Feature Selector	threshold = 0.1	threshold = 0.05	Kam aggressive pruning — halke-se useful features bhi rakhe jaate hain (chunki XGBoost khud bhi weak features ko largely ignore kar sakta hai apne regularization se)
5	Target	Raw price	<code>log1p(price)</code> transform	Price distribution right-skewed thi (EDA mein dekha) — log transform isse zyada symmetric/Gaussian banata hai, jisse model ka loss function better converge karta hai aur bade outlier prices ka asar kam hota hai

`is_north()` function:

```
def is_north(X):
    north_cities = ["Delhi", "Kolkata", "Mumbai", "New Delhi"]
    return X.assign(**{f"{col}_is_north": X[col].isin(north_cities).astype(int) for col in X.columns}).drop(columns=north_cities)
```

`is_peak_hour()` function:

```
def is_peak_hour(X):
    hour = pd.to_datetime(X[col]).dt.hour
    result[f"{col}_is_peak"] = (hour.between(6,9,inclusive="both") | hour.between(17,20,inclusive="both")).astype(int)
```

Baaki sab (airline, additional_info, total_stops transformers) **exactly same** rahe jaise baseline mein the.

Preprocessor ko Log-Target par Fit Karna

```
preprocessor.fit(train.drop(columns="price"), np.log1p(train.price))
```

Kyun important: Feature Selector (`SelectBySingleFeaturePerformance`) internally target `y` ka use karke har feature ka R^2 score nikalta hai. Agar preprocessor ko raw `price` par fit karte aur model ko `log1p(price)` par train karte, to feature-selection ka "usefulness score" aur actual model ka target thoda mismatch ho sakta tha. Isliye **end-to-end consistency** ke liye poore pipeline ko `log1p(price)` target ke against hi fit kiya gaya.

Data ko S3 ke liye Export karna

```
def export_data(data, name, pre):
    X_pre = pre.transform(data.drop(columns="price"))
    y = np.log1p(data.price)
```

```
y.to_frame().join(X_pre).to_csv(f"{name}-pre.csv", index=False, header=False)
```

SageMaker ke built-in XGBoost algorithm ka strict data format requirement hota hai: - CSV file mein **koi header row nahi** honi chahiye. - **Target column sabse pehle** honi chahiye, uske baad features.

Isliye `y.to_frame().join(X_pre)` se target ko pehle column mein rakha, phir `header=False, index=False` se save kiya. Phir `boto3` (AWS SDK for Python) se `s3://bucket/data/{train,val,test}/{name}.csv` par upload kiya.

SageMaker Estimator Setup

```
model = Estimator(
    image_uri=sagemaker.image_uris.retrieve("xgboost", region_name, "1.2-1"),
    role=sagemaker.get_execution_role(),
    instance_count=1,
    instance_type="ml.m5.xlarge",
    volume_size=5,
    output_path=output_path,
    use_spot_instances=True,
    max_run=600, max_wait=1200
)
```

- **image_uri** — AWS ka pre-built Docker container jisme XGBoost already installed hai (version 1.2-1) — apna khud ka training script likhne ki zaroorat nahi (SageMaker "built-in algorithm" mode).
- **role** — IAM role jo SageMaker ko S3 access aur baaki AWS resources access karne ki permission deta hai.
- **instance_type="ml.m5.xlarge"** — general-purpose EC2 instance (4 vCPU, 16GB RAM), medium-size tabular data ke liye kaafi hai.
- **use_spot_instances=True** — AWS ki **unused/spare compute capacity** ko discount pe (~70-90% cheaper) use karta hai. Trade-off: instance kabhi bhi AWS dwara reclaim ho sakta hai (interrupt), isliye `max_wait` ek buffer deta hai retry ke liye.

Base Hyperparameters

```
model.set_hyperparameters(
    objective="reg:squarederror", num_round=200, eta=0.05, max_depth=6,
    min_child_weight=3, subsample=0.8, colsample_bytree=0.8, alpha=0.1,
    early_stopping_rounds=20
)
```

Parameter	Value	Matlab
<code>objective</code>	<code>reg:squarederror</code>	Regression loss = Mean Squared Error minimize karna. (Purana <code>reg:linear</code> deprecated ho chuka hai — readme mein specifically mention hai)
<code>num_round</code>	200	Kitne boosting rounds/trees banaye jayein

<code>eta</code>	0.05	Learning rate — chhota value = har tree ka contribution kam, model dheere-dheere seekhta hai (zyada conservative, overfitting kam)
<code>max_depth</code>	6	Har tree ki max depth (complexity control)
<code>min_child_weight</code>	3	Ek leaf/child node mein minimum instances ka weight — chhote/overly-specific splits ko rokta hai
<code>subsample</code>	0.8	Har tree sirf 80% random rows par train hota hai (stochastic — variance kam karta hai)
<code>colsample_bytree</code>	0.8	Har tree sirf 80% random features use karta hai (trees ko decorrelate karta hai)
<code>alpha</code>	0.1	L1 regularization (leaf weights ko sparse/chhota rakhta hai)
<code>early_stopping_rounds</code>	20	Agar 20 rounds tak validation metric improve na ho, training rok do (overfitting/waste-of-compute se bachata hai)

Bayesian Hyperparameter Tuning

```
hyperparameter_ranges = {
    "eta": ContinuousParameter(0.01, 0.2), "alpha": ContinuousParameter(0, 2),
    "lambda": ContinuousParameter(0, 2), "max_depth": IntegerParameter(4, 8),
    "min_child_weight": IntegerParameter(1, 10), "subsample": ContinuousParameter(0.6, 1.0),
    "colsample_bytree": ContinuousParameter(0.6, 1.0), "gamma": ContinuousParameter(0, 5)
}
tuner = HyperparameterTuner(
    estimator=model, objective_metric_name="validation:rmse",
    hyperparameter_ranges=hyperparameter_ranges, strategy="Bayesian",
    objective_type="Minimize", max_jobs=20, max_parallel_jobs=2
)
tuner.fit(data_channels)
```

Bayesian Optimization kya hai (bahut common interview question): Ye ek smart search strategy hai jo **pichle trials ke results se seekhta hai** — ek internal probabilistic surrogate model (jaise Gaussian Process) banata hai jo predict karta hai "kaunsa naya hyperparameter combination try karne se best result milega" (exploration vs exploitation balance karta hai). Ye **Grid Search** (sab combinations try karo — bahut slow) aur **Random Search** (random try karo — koi learning nahi) se zyada **efficient** hota hai kyunki har naya trial pichle trials se guided hota hai.

- **max_jobs=20, max_parallel_jobs=2** — total 20 training jobs challenge cloud pe, lekin ek time pe sirf 2 parallel. Kam parallelism isliye rakha gaya kyunki Bayesian method sequential learning se fayda uthata hai (agar sab jobs ek saath parallel chalen to optimizer ko beech mein "seekhne" ka mauka kam milta hai).

- **objective_metric_name="validation:rmse", minimize** — RMSE (Root Mean Squared Error) validation set par sabse kam ho, yehi best model chuna jaata hai.

Best Model Nikalna

```
best_estimator = tuner.best_estimator()
# S3 se model.tar.gz download, extract kiya
with open("xgboost-model", "rb") as f:
    best_model = pickle.load(f)
best_model.save_model("modern_xgboost_model.json")
```

SageMaker automatically saare 20 jobs mein se best-performing (lowest validation RMSE) wala model dhoondh leta hai, uska artifact (**model.tar.gz**) S3 se download karke pickle se load kiya, phir XGBoost ke **native JSON format** mein save kiya.

JSON format kyun (pickle ki jagah)? Pickle Python/library version ke saath tightly-coupled hota hai — agar training environment (SageMaker's XGBoost 1.2-1 container) aur deployment environment (Streamlit Cloud) mein slight version difference ho, toh pickle load karte waqt error/crash ho sakta hai. XGBoost ka JSON format **version-independent aur forward-compatible** hai, isliye production deployment ke liye safe hai.

Model Evaluation

```
def evaluate_model(split):
    X = preprocessor.transform(data.drop(columns="price").values)
    pred_log = best_model.predict(xgb.DMatrix(X))
    pred = np.expml(pred_log) # log-space se wapas real price mein
    r2 = r2_score(data["price"].values, pred)
```

Important: Model ne **log1p(price)** predict karna seekha tha, isliye prediction bhi log-space mein aata hai — **np.expml()** (jo **log1p** ka exact inverse hai: **expml(x) = e^x - 1**) se wapas real ₹ price mein convert kiya jaata hai, **tabhi** R² real price scale par calculate hota hai.

Final Results: | Split | R² Score | ---|---| | Train | **0.9085** | | Validation | **0.7906** | | Test | **0.7921** |

Interpretation (interview mein ye gap zaroor poocha jaayega): Train (0.91) aur Val/Test (~0.79) ke beech ~0.12 ka gap hai — ye **mild-to-moderate overfitting** ki taraf ishara karta hai (regularization — subsample, colsample, alpha/lambda, gamma, early stopping — hone ke bawajood, kyunki dataset chhota hai ~6700 rows aur features rich/engineered hain, model training data ke patterns ko kuch had tak "yaad" kar leta hai). Lekin **Val ≈ Test** (0.79 dono) ek **achha sign** hai — iska matlab hai ki tuning process ne validation set ko "overfit" nahi kiya (agar aisa hota to test score val se kaafi kam hota), model genuinely generalize kar raha hai naye data par.

PART 5 — **app.py** (Streamlit Web App — Deployment)

Ye file **poore preprocessing pipeline ka code dobara copy-paste karti hai** (**is_north**, **is_peak_hour**, 5 RBF anchors, threshold=0.05 — exactly wahi jo TrainingData.ipynb mein tha) directly script ke andar, instead of **preprocessor.pkl** ko load karne ke.

Ye design choice kyun (bahut acha interview/MLOps discussion point)?

Problem: Agar tu ek **custom Python class** (jaise `RBFPPercentileSimilarity`) ko pickle karke save karta hai, to use wapas load (`pickle.load`) karne ke liye **exact wahi class definition** available honi chahiye, exact matching library versions ke saath. Agar Streamlit Cloud ka environment (sklearn/feature-engine version) training environment se thoda bhi mismatch ho, to **pickle load fail ho sakta hai** — ye classic MLOps pitfall hai.

Solution jo yahan use hua: App startup par preprocessor ko **fresh refit** karo:

```
train = pd.read_csv("Data/train.csv")
X_train = train.drop(columns="price")
y_train = train.price.copy()
preprocessor.fit(X_train, np.log1p(y_train)) # live refit, pickle load nahi
joblib.dump(preprocessor, "preprocessor.joblib")
```

Isse deployment environment ke apne hi library versions ke saath preprocessor consistent rehta hai (koi cross-version incompatibility risk nahi) — trade-off ye hai ki app start hone mein thoda extra time lagta hai, aur code do jagah maintain karna padta hai (DRY principle violate hota hai — agar training pipeline update ho aur app.py mein sync na kiya jaaye to dono mismatch ho sakte hain — code mein "FIX" comments isi manual-sync process ka evidence hain).

Model (XGBoost weights) alag treat hota hai: Model ko retrain nahi kiya jaata (wo expensive hai, ek baar SageMaker par ho chuka), sirf portable `xgboost-model.json` load hota hai:

```
model = xgb.Booster()
model.load_model("xgboost-model.json")
```

UI Layout

```
st.set_page_config(page_title="Flight Price Prediction", page_icon="", layout="wide")
col1, col2, col3 = st.columns(3)
```

3-column form: Col1 = Airline/Source/Destination dropdowns (`X_train` ke unique values se populate), Col2 = Date/Dep-Time/Arrival-Time inputs, Col3 = Duration/Total-Stops/Additional-Info.

Prediction Flow

```
x_new = pd.DataFrame(...).astype({col: "str" for col in ["date_of_journey", "dep_time", "arrival_time"]})

if st.button("Predict Price"):
    x_new_pre = preprocessor.transform(x_new)
    x_new_xgb = xgb.DMatrix(x_new_pre.values)
    pred_log = model.predict(x_new_xgb)[0]
    pred = np.expml(pred_log)
    st.success(f"### Predicted Price: ₹{pred:,.0f} INR")
```

- `.astype(str)` zaroori hai kyunki Streamlit ke `date_input/time_input` widgets Python native `date/time` objects return karte hain, lekin trained pipeline ke andar `pd.to_datetime()/DatetimeFeatures` string input expect karte hain (training-time format se match karne ke liye).
- `preprocessor.transform()` (fit nahi!) — kyunki encoders/scalers pehle se `X_train` par fit ho chuke hain; naya single-row input sirf unhi learned statistics se transform hota hai (**no leakage** — bilkul waisa hi jaisa eval mein train/val/test transform hote hain).

- `xgb.DMatrix` — XGBoost ka optimized internal data structure (memory-efficient, C++ backend ke saath compatible).
- `np.expml()` — log-space prediction ko wapas real ₹ price mein convert karta hai.
- `try/except` — koi bhi error (jaise unseen category, bad input) gracefully `st.error()` mein show hoti hai, app crash nahi hota.

PART 6 — `convert.py` (Utility Script)

```
with open("xgboost-model", "rb") as f:
    model = pickle.load(f)
    model.save_model("xgboost-model.json")
```

Ye ek **standalone one-off utility** hai — SageMaker training se aaye raw pickled Booster (`xgboost-model` file, jo `model.tar.gz` ke andar tha) ko load karke, XGBoost ke native **JSON format** mein wapas save karta hai. Ye wahi conversion hai jo TrainingData.ipynb ke andar bhi hoti hai — is script ka fayda ye hai ki isse kabhi bhi standalone (bina poora notebook re-run kiye) re-run kiya ja sakta hai agar sirf format-conversion dobara karna ho.

PART 7 — 200 Interview Questions (with Answers)

Questions English mein hain (jaisa actual interview mein poochha jaayega), answers concise rakhe hain — exam se pehle revise karne ke liye perfect. Har section tere project ke ek specific hisse se directly linked hai.

Section A — Statistics & Hypothesis Testing (Q1-45)

Q1. What is the difference between Pearson and Spearman correlation? Pearson measures *linear* correlation between two continuous variables and assumes roughly normal, outlier-free data. Spearman measures *monotonic* (rank-based) correlation — it converts values to ranks first, so it's robust to outliers and captures non-linear-but-monotonic relationships.

Q2. Why did you use Spearman in your correlation heatmap instead of Pearson? Because features like `duration` and `price` are right-skewed with outliers, and their relationship isn't strictly linear. Spearman gives a more honest picture of association strength in that situation.

Q3. Your Spearman correlation (0.698) between duration and price was higher than Pearson (0.509). What does that tell you? It signals the relationship is monotonic but non-linear — as duration increases price tends to increase, but not at a constant rate. This directly justified using RBF kernel similarity features instead of just a linear scaled duration value.

Q4. What is Cramér's V and why use it over a raw chi-square statistic? Cramér's V is a normalized measure (0 to 1) of association between two categorical variables, derived from the chi-square statistic but scaled by sample size and table dimensions so it's comparable across different

variable pairs — like a "correlation coefficient" for categoricals.

Q5. Explain the bias correction applied in the Cramér's V formula you implemented. Raw Cramér's V is upward-biased for small samples. The correction (Bergsma's) subtracts an expected-bias term from ϕ^2 and adjusts row/column counts before taking the square root, so the metric doesn't overstate association purely due to sample size / table shape.

Q6. What is the Shapiro-Wilk test used for? It tests the null hypothesis that a sample comes from a normally distributed population. A very small p-value (as you got, $\sim 7.4e-57$ for duration) means you reject that null — the data is not normal.

Q7. How is Anderson-Darling different from Shapiro-Wilk? Anderson-Darling gives more weight to the tails of the distribution, making it more sensitive to deviations at the extremes, whereas Shapiro-Wilk is generally considered the most powerful all-round normality test for small-to-medium samples.

Q8. Why run two normality tests instead of just one? As a robustness check — different tests have different sensitivities/assumptions, so agreement between them (both rejected normality here) gives more confidence in the conclusion.

Q9. What is a p-value, in plain terms? It's the probability of observing data at least as extreme as what you got, *assuming the null hypothesis is true*. A small p-value means your observed data would be very unlikely under the null, so you reject it.

Q10. Difference between Type I and Type II error? Type I error = rejecting a true null hypothesis (false positive). Type II error = failing to reject a false null hypothesis (false negative). Significance level (α , usually 0.05) controls the Type I error rate.

Q11. Duration had skewness = 0.89. What does that mean? Positive/right skew — the distribution has a long tail toward higher values, so mean (633.5) > median (510); a few very long-duration flights pull the average up.

Q12. What does kurtosis ≈ -0.05 tell you? Kurtosis near 0 (excess kurtosis, relative to normal) means the tails/peakedness of the distribution are close to normal — it's the skewness, not the tail-heaviness, that's the bigger deviation from normality here.

Q13. What is a trimmed mean, and why compute it alongside the regular mean? Trimmed mean removes a percentage (e.g. 5%/10%) of extreme values from both ends before averaging — it's more robust to outliers. Comparing it to the regular mean shows how much outliers are skewing your average.

Q14. Explain the IQR method for outlier detection. $IQR = Q3 - Q1$. Any point below $Q1 - 1.5 \times IQR$ or above $Q3 + 1.5 \times IQR$ is flagged as an outlier. You used this to find duration's bounds (-958.75 to 2051.25 minutes).

Q15. Difference between IQR-based outlier detection and Isolation Forest? IQR is univariate — it looks at one column at a time. Isolation Forest is multivariate/model-based — it considers combinations of features together to flag points that are jointly unusual, which IQR on a single column would miss.

Q16. How does Isolation Forest work internally? It builds many random trees that recursively split data on random features/thresholds. Anomalies get isolated (end up alone in a leaf) in fewer splits than normal points, since they're "different" along some dimension — so average path length to isolation becomes the anomaly score.

Q17. Why is Isolation Forest called unsupervised? Because it doesn't need labeled "outlier"/"normal" examples — it learns the notion of "unusual" purely from the structure of the data itself.

Q18. What is ANOVA (F-test), and what does it assume? ANOVA tests whether the means of 3+ groups are statistically equal. It assumes normality within groups and roughly equal variances (homoscedasticity) — assumptions your price data likely violates since it's skewed.

Q19. Why did you also run Kruskal-Wallis alongside ANOVA for airline vs price? Kruskal-Wallis is ANOVA's non-parametric counterpart — it compares group *medians/rank distributions* and doesn't assume normality. Since price violates normality, running it as a cross-check makes the "airlines differ significantly" conclusion more trustworthy.

Q20. State the null and alternate hypotheses for the airline-vs-price test. H0: all airlines have the same population mean (or median, for Kruskal-Wallis) price. H1: at least one airline's price differs. Both tests gave $p \approx 0$, so H0 was rejected.

Q21. Difference between parametric and non-parametric tests? Parametric tests (t-test, ANOVA, Pearson) assume a specific data distribution (usually normal) and use its parameters (mean, variance). Non-parametric tests (Kruskal-Wallis, Spearman, Mann-Whitney) make fewer distributional assumptions, working on ranks instead.

Q22. Explain the Chi-square test of independence and how it's used for categorical-categorical association. It compares observed frequency counts in a contingency table against expected counts under the assumption the two variables are independent. A large chi-square statistic (small p-value) means the variables are associated, not independent — used as the basis for Cramér's V.

Q23. Why use MAD (median absolute deviation) instead of standard deviation? Standard deviation is sensitive to outliers (squares deviations). MAD uses the median of absolute deviations from the median, which is far more robust — useful here since duration has extreme values.

Q24. What's coefficient of variation, and why compute it? $CV = \text{std} / \text{mean}$ — a unit-less measure of relative spread, useful for comparing variability across variables/datasets with different scales.

Q25. What is a QQ (quantile-quantile) plot used for? It plots your data's quantiles against theoretical normal-distribution quantiles. If points fall roughly on the 45° line, data is normal; deviations (especially curving away at the tails) indicate skew or heavy tails — exactly what you'd expect to see for duration.

Q26. What does "significance level = 0.05" mean in practice? It means you're willing to accept a 5% chance of incorrectly rejecting a true null hypothesis (Type I error) — it's the threshold you compare your p-value against.

Q27. If $p = 0.049$, is the result "definitely" significant? What's the pitfall of hard $p < 0.05$ thresholding? Technically yes by convention, but 0.049 vs 0.051 is a razor-thin, somewhat arbitrary distinction — over-relying on a hard cutoff ignores effect size and can lead to "p-hacking" mentality. Better practice reports the actual p-value and effect size together.

Q28. What's the difference between correlation and causation? Give an example from your project. Correlation just means two variables move together; it doesn't prove one causes the other. E.g., "peak hour" flights being pricier doesn't necessarily mean the *time itself* causes higher price — it could be confounded by which airlines/routes fly at those hours.

Q29. Why check both Pearson and Spearman instead of picking one? They test slightly different things (linear vs monotonic) and have different sensitivities to outliers/non-linearity — running both gives a fuller picture and cross-validates the conclusion of "these variables are associated."

Q30. What assumption does Pearson make that Spearman doesn't? Pearson assumes the relationship is linear and is sensitive to outliers (since it works on raw values); Spearman only assumes monotonicity and works on ranks, so it's distribution-free and outlier-robust.

Q31. How were percentiles used as "reference points" in your RBF duration feature? The 10th/25th/50th/75th/90th percentile duration values became fixed "anchor points." Every flight's duration was then scored on how similar (Gaussian-kernel-wise) it was to each anchor — giving a smooth positional signal instead of a hard bucket.

Q32. Explain z-score standardization mathematically. $z = (x - \mu) / \sigma$. It rescales a feature to have mean 0 and standard deviation 1, so different features become comparable in scale. Used via `StandardScaler` on duration.

Q33. What's positive (right) skew vs negative (left) skew, and how would you typically fix it? Right skew = long tail toward high values (mean > median), like your duration/price. Left skew is the mirror case. Common fixes: log/log1p transform, Box-Cox, or Yeo-Johnson (PowerTransformer) — all used somewhere in this project.

Q34. Why is it reasonable to assume price is also right-skewed, even though you didn't explicitly test it? Real-world prices almost always have a long right tail (a few very expensive flights), and this directly motivated using `log1p(price)` as the model's training target to make the distribution more symmetric.

Q35. If you wanted to compare price between exactly two specific airlines (not all), what test would you use? A two-sample test: independent t-test if roughly normal/equal variance, or the non-parametric Mann-Whitney U test if not — analogous to how ANOVA/Kruskal-Wallis generalize to 2+ groups.

Q36. What's a confidence interval, and how does it relate to hypothesis testing? A range of plausible values for a population parameter (e.g., mean) at a given confidence level (e.g., 95%). A 95% CI that excludes the null-hypothesized value is roughly equivalent to rejecting H_0 at $\alpha=0.05$.

Q37. What is multicollinearity, and how did you (informally) check for it? When two or more features are highly correlated with each other, which can destabilize coefficient estimates in linear models. The Spearman/Cramér's V heatmaps served as an informal check (though for tree-based XGBoost, multicollinearity is less harmful than for linear regression).

Q38. Difference between covariance and correlation? Covariance measures the direction of a linear relationship but its magnitude depends on the variables' units/scale. Correlation normalizes covariance by the product of standard deviations, giving a scale-free value between -1 and 1.

Q39-40. What are MCAR, MAR, MNAR missing-data mechanisms, and which applies to your single missing `total_stops` value? MCAR = missing completely at random (no pattern). MAR = missingness depends on other observed variables. MNAR = missingness depends on the missing value itself. With just 1 missing value out of 6695, there's no strong evidence either way, so it was reasonably treated as MCAR and filled with the mode (0 stops).

Q41. Why fill the missing `total_stops` with 0 rather than drop the row? Only 1 row was affected (~0.015%) — dropping vs imputing makes negligible statistical difference, but imputing preserves sample size, and 0 (non-stop) was the safest choice since it was also the most-frequent-adjacent, defensible value.

Q42. What is bootstrapping, and could you use it here? Repeatedly resampling your dataset (with replacement) to estimate the sampling distribution of a statistic. You could use it to put a confidence interval around your R^2 score rather than reporting a single point estimate.

Q43. Why is the Central Limit Theorem relevant even when your raw data isn't normal? CLT says the *sampling distribution of the mean* approaches normal as sample size grows, regardless of the underlying population's shape — this is partly why ANOVA (an F-test on means) can still be reasonably applied on $n=6695$, even though individual values are skewed. (Though you correctly cross-checked with the non-parametric Kruskal-Wallis anyway.)

Q44. What are "degrees of freedom" in a chi-square or ANOVA test? Roughly, the number of independent values that can vary given constraints (like fixed row/column totals in a contingency table, or number of groups minus 1 in ANOVA). It determines the shape of the reference distribution used to compute the p-value.

Q45. Why is statistical significance not the same as practical/business significance? With a large sample size ($n=6695$), even a tiny, practically meaningless difference can produce a very small p-value. You should always look at effect size (e.g., how many rupees difference in price) alongside the p-value, not just "significant or not."

Section B — Data Cleaning & Pandas (Q46-65)

Q46. Why `dayfirst=True` when parsing `Date_of_Journey`? Indian date format is DD/MM/YYYY. Without `dayfirst=True`, pandas could misinterpret something like "03/04/2019" as March 4th (US-style MM/DD) instead of April 3rd.

Q47. Walk through how you converted "2h 50m" into total minutes. Split the string on space into hour/minute parts, strip the "h"/"m" letters, convert to int, multiply the hour part by 60, and sum hour-in-minutes + minutes.

Q48. What edge cases did that conversion need to handle? Some durations had only hours (e.g. "19h", no minute part) and one had only minutes ("5m", no hour part) — `fillna("0")` on the minute column handled the missing-minute case; the missing-hour case ("5m") turned out to be a data-entry error and was dropped instead.

Q49. How did you find and justify dropping row 6474? While inspecting duration values that didn't contain "h", you found one row with duration = "5m" only — a flight can't realistically complete in 5 minutes, so this was flagged as corrupted data and explicitly dropped by index.

Q50. How did you detect the 220 duplicate rows? `flights.duplicated().sum()` — pandas compares all column values row-wise and flags exact repeats.

Q51. Why is removing duplicates important before modeling? Duplicate rows give certain data points disproportionate influence during training (effectively upweighting them), which can bias the model and inflate apparent performance if duplicates leak across train/test splits.

Q52. What does `.pipe()` do in pandas? It lets you pass a DataFrame through a custom function as part of a method chain, e.g. `df.pipe(my_func)`, keeping code readable in a fluent chained style instead of nesting function calls.

Q53. Difference between `.assign()` and direct column assignment (`df['col'] = ...`)? `.assign()` returns a new DataFrame (doesn't mutate in place) and works well inside method chains; direct assignment mutates the original DataFrame. `.assign()` is preferred in the functional/chained style used throughout this project.

Q54. Why drop the `Route` column entirely instead of engineering it? It was redundant with source+destination for this version of the pipeline, and encoding a variable-length sequence of connecting airports adds real complexity — the project scoped it out for simplicity (a valid future improvement, though).

Q55. What could you have done with `Route` instead of dropping it? Extracted features like number of connecting airports (correlates with stops, though somewhat redundant), or specific hub-airport indicators, or route-frequency encoding.

Q56. Why was `pd.to_datetime(...).dt.time` needed for `Arrival_Time`? Some arrival times included a date suffix (e.g., "01:10 22 Mar", for overnight flights landing the next day). Parsing to full

datetime and taking `.dt.time` cleanly extracts just the time-of-day, discarding the extra date info.

Q57. What is method chaining, and what are its pros/cons? Calling a sequence of DataFrame methods one after another (`.assign().rename().drop()...`) instead of reassigning intermediate variables. Pros: readable, mirrors a data "pipeline," avoids intermediate variable clutter. Cons: harder to debug mid-chain, and stack traces can be less clear.

Q58. Why lowercase all column names? Consistency and convention — avoids bugs from case-sensitivity (`Airline` vs `airline`) and makes code cleaner across the whole project, since every downstream notebook/script relies on lowercase names.

Q59. Beyond what was done here, how would you robustly handle a column with genuinely mixed data types in production? Add explicit schema validation (e.g., Pydantic/pandera), log/quarantine rows that fail type coercion instead of silently erroring, and add unit tests for the cleaning function on edge-case inputs.

Q60. What does the regex `" stops?"` in `.str.replace(" stops?", "", regex=True)` mean? The `?` makes the preceding "s" optional, so it matches both " stop" (singular) and " stops" (plural) in one pattern — a compact way to normalize both forms.

Q61. Why was the airline text normalized (removing "Premium economy"/"Business" suffixes)? These suffixes created spurious extra categories (e.g., "Vistara" vs "Vistara Premium economy") for what's really the same core airline — merging them reduces unnecessary cardinality/noise.

Q62. Why title-case the airline names? For consistent, human-readable formatting and to avoid case-based duplicate categories (e.g., "AIR INDIA" vs "Air India" being treated as different labels).

Q63. What's the risk of silently `.fillna()` without first understanding why data is missing? You could mask a real data-quality/collection issue, introduce bias if the mechanism isn't actually MCAR, or hide a bug in an upstream pipeline. Always ideally check the missing pattern/percentage first (which was done here — the missing count was inspected before imputing).

Q64. Why check `flights.dtypes` right at the start, before doing any deeper analysis? It's the fastest way to catch structural issues (e.g., everything read in as `object` when some columns should be numeric/datetime) — informs the whole cleaning plan.

Q65. What would you change about this cleaning process if the dataset were 10x larger (millions of rows)? Avoid pandas alone (which is single-machine, in-memory) — consider chunked processing, Dask/PySpark for distributed cleaning, and vectorized regex operations over `.apply()` for speed.

Section C — EDA & Visualization (Q66-80)

Q66. Why do EDA before feature engineering at all? To understand distributions, relationships, and data quality issues *before* deciding how to transform/encode features — e.g., the skewness and non-linear duration-price relationship directly informed the choice of RBF kernel features and log1p target transform later.

Q67. What's the purpose of a pair plot, and why only plot the lower triangle? It shows scatterplots between every pair of numeric variables at a glance, to eyeball possible relationships. The upper triangle is a mirror image of the lower triangle, so plotting both is redundant.

Q68. Why use a hexbin plot in addition to a scatter plot for duration vs price? With thousands of points, a scatter plot gets overplotted (points overlapping, hard to see density). A hexbin plot bins the 2D space into hexagons and colors by count, making density patterns visible even with

overplotting.

Q69. What is `ydata_profiling` (formerly `pandas-profiling`), and why generate that report? An automated EDA library that produces a comprehensive HTML report (distributions, correlations, missing values, warnings) with one line of code — used here as a fast, broad cross-check alongside the more targeted manual analysis.

Q70. Why bin `dep_time` into morning/afternoon/evening/night categories for some analyses? Raw hour-of-day (0-23) is high-cardinality and treats each hour as equally distinct; bucketing into broader "parts of day" can reveal coarser patterns more clearly in visualizations, and became a modeling feature too (`CountFrequencyEncoder`).

Q71. Why use a word cloud for a categorical variable like `additional_info`? It's a quick, intuitive visual of which category values dominate by frequency (bigger text = more frequent) — more for stakeholder-friendly communication than for rigorous analysis.

Q72. What would you look for in a box plot of price grouped by airline? The median line (typical price), the box height (IQR/spread), whisker length, and outlier points beyond the whiskers — to compare both central tendency and variability across airlines at a glance.

Q73. Why plot monthly AND weekly average price trends for `date_of_journey`? Monthly view shows broad seasonality (e.g., summer travel demand), while weekly view can reveal finer-grained fluctuations (e.g., specific busy weeks/holidays) that a monthly aggregate would smooth over.

Q74. What does a violin plot show that a box plot doesn't? A violin plot overlays a full density/distribution shape (like a mirrored KDE) on top of the box plot summary stats — so you can see multi-modality or distribution shape, not just quartiles.

Q75. Why examine `additional_info`'s category distribution before deciding how to encode it? It revealed severe class imbalance (78% "No Info," several categories under 1%) — that finding directly justified adding a coarser binary `have_info` flag alongside the rare-label-grouped one-hot encoding, rather than just one-hot encoding all 7 raw categories.

Q76. What's the danger of one-hot encoding a category that appears only once or twice in your data? It creates a near-useless, extremely sparse column that the model can easily overfit to (memorizing that one specific row) without any real generalizable signal.

Q77. How would you check if EDA findings generalize to val/test, not just train? Compare summary statistics (mean, distribution shape, category proportions) of val/test against train — large discrepancies would indicate the split isn't representative or there's some data drift.

Q78. Why is it good practice to do EDA only on the training set, not on val/test? To avoid information leakage — decisions informed by peeking at val/test data (even just visually) can subtly bias modeling choices in ways that make test performance look artificially better than it would be on truly unseen data.

Q79. If you found a strong bivariate relationship in EDA (like duration-price), does that guarantee the model will use it well? No — EDA finds *associations*, but whether the model actually exploits them well depends on how the feature is encoded/transformed and whether feature selection keeps it (which duration passed, with a very high standalone R^2 of 0.425).

Q80. What's one EDA finding you'd flag as a limitation or risk for the final model? The severe class imbalance in `additional_info` and the rarity of some airlines — the model has very little data to learn genuinely reliable patterns for those rare categories, so predictions involving them are inherently less trustworthy.

Section D — Feature Engineering & Encoding (Q81-125)

Q81. What is one-hot encoding, and when is it appropriate? Converts each category into its own binary (0/1) column. Appropriate for low-to-moderate cardinality nominal (unordered) categorical variables, like airline (after rare-label grouping) here.

Q82. What is Rare Label Encoding, and why did you apply it before one-hot encoding? It groups categories below a frequency threshold (`tol`) into a single "Other" bucket. Applying it before one-hot encoding keeps the number of dummy columns small and avoids creating columns for categories with almost no data (e.g., Trujet, which appeared once).

Q83. What does `n_categories=2` do in `RareLabelEncoder`? It guarantees at least the top-2 most frequent categories are always kept separate, even if one of them happens to fall slightly under the `tol` threshold — a safety floor.

Q84. What is Mean/Target Encoding, and why is it powerful for high-cardinality features like source/destination? It replaces each category with the mean of the target variable for that category (e.g., average price for flights from Bangalore). Unlike one-hot, it doesn't explode dimensionality and it directly encodes the category's relationship with the target.

Q85. What's the main risk of Mean/Target Encoding, and how would you properly mitigate it? Data leakage/overfitting — a row's own target value contributes to its own category's mean, especially problematic with small groups. Proper mitigation uses cross-fold (out-of-fold) target encoding or adds smoothing toward the global mean for low-count categories, rather than a simple one-shot leave-in fit as used here.

Q86. Why apply `PowerTransformer` after mean encoding? To make the resulting encoded values more Gaussian-like / reduce skew — can help downstream models (especially non-tree ones) and stabilizes the numeric scale of these mean-encoded features.

Q87. What's the difference between `PowerTransformer` and `StandardScaler`? `StandardScaler` just re-centers/rescales (linear transform, mean 0, std 1) without changing distribution shape. `PowerTransformer` (Yeo-Johnson/Box-Cox) applies a non-linear transform to actually reshape a skewed distribution toward normality.

Q88. What's the difference between Box-Cox and Yeo-Johnson (both inside `PowerTransformer`)? Box-Cox requires strictly positive input values. Yeo-Johnson (sklearn's default) extends the idea to handle zero and negative values too — important here since mean-encoded price differences or some engineered features could be zero/negative.

Q89. Why use `MinMaxScaler` for date-of-journey features but `StandardScaler` for duration? Somewhat a design/consistency choice more than a strict necessity for tree models — `MinMax` keeps date-derived features (month, week, etc.) bounded in [0,1], while duration also gets a separate power-transformed and RBF view, so `StandardScaler` there just preserves one "plain" standardized version of the raw signal.

Q90. What is `CountFrequencyEncoder`, and how does it differ from `OrdinalEncoder`? `CountFrequencyEncoder` replaces each category with its occurrence count/proportion in the training data — no arbitrary rank order is imposed. `OrdinalEncoder` assigns integer ranks (0,1,2...) which implies a specific order — appropriate only when a real order exists (like `duration_cat`: short<medium<long), not for something like "part of day."

Q91. Explain the RBF (Gaussian) kernel similarity feature in detail. For each duration value x , and each percentile "anchor" a , it computes `similarity = exp(-gamma(x-a)^2)`. This gives a smooth score of "how close is x to this reference point," bounded between 0 and 1, peaking at 1 when x equals the anchor and decaying smoothly as distance grows.

Q92. Why use 5 percentile anchors instead of, say, 10 or 3? More anchors capture the distribution's shape more finely (especially the tails, with 10th/90th percentile anchors added) but add more columns/complexity; 3 was the initial simple baseline, 5 was found to be a reasonable

improvement without excessive dimensionality — a hyperparameter that could itself be tuned.

Q93. What does the `gamma` parameter control in the RBF kernel? It controls how quickly similarity decays with distance — small `gamma` = broad/smooth similarity (far values still considered somewhat similar), large `gamma` = sharp/localized similarity (only very close values count as similar).

Q94. Why not just bin duration into simple categories instead of using RBF similarity? Hard bins create abrupt discontinuities at boundaries (e.g., 179 vs 181 minutes suddenly jump categories), losing information about *how close* a value is to a boundary. RBF similarity gives a smooth, continuous alternative that still captures "closeness to typical durations."

Q95. What is Winsorizer, and how is it different from simply dropping outlier rows? Winsorizer caps (clips) extreme values at a boundary (e.g., the $1.5 \times \text{IQR}$ limit) instead of removing the row entirely — this reduces the outlier's influence on scaling/statistics while preserving the sample size and not discarding potentially useful information from the rest of that row's other features.

Q96. Why use median imputation for duration instead of mean? Duration is right-skewed, so the mean is pulled upward by extreme values; median is robust to skew and represents a more "typical" duration.

Q97. Explain `FeatureUnion` vs `ColumnTransformer` — when do you use each? `ColumnTransformer` applies *different* transformers to *different* columns and concatenates results (e.g., one pipeline for airline, another for duration). `FeatureUnion` applies *multiple* transformers to the *same* column(s) and concatenates their outputs side by side (e.g., getting both RBF-similarity and category-binning views of duration).

Q98. What does `sklearn.set_config(transform_output="pandas")` do, and why use it? It makes sklearn transformers return pandas DataFrames (with meaningful column names) instead of raw numpy arrays — makes debugging and inspecting intermediate pipeline outputs much easier.

Q99. Why is `handle_unknown="ignore"` important in the `OneHotEncoder` used here? In production, a brand-new category (e.g., a new airline not seen in training) could appear. Without this setting, the encoder would throw an error; with it, the unseen category is safely encoded as all-zeros instead of crashing the app.

Q100. What is the `is_north` feature, and what's the underlying hypothesis? A binary flag = 1 if source/destination is a "northern"/major metro city (Delhi, Kolkata, Mumbai, New Delhi). The hypothesis is that route geography/hub status correlates with pricing patterns differently than smaller cities like Cochin/Chennai/Bangalore.

Q101. What is the `is_peak_hour` feature, and why 6-9AM and 5-8PM specifically? A binary flag capturing typical commute/high-demand travel windows — the hypothesis is flights during these busy hours are priced at a premium due to higher demand. The specific hours are a reasonable domain-knowledge choice (could be tuned/validated further with data).

Q102. Why create BOTH a one-hot encoding AND a binary "`have_info`"/"`is_direct`"/"`is_north`" flag for the same underlying information? Different granularities of the same signal help the model in different ways — the fine-grained one-hot lets it learn category-specific effects (if data supports it), while the coarse binary flag gives a robust fallback signal even when individual categories are too rare to learn reliably.

Q103. Why is `remainder="passthrough"` used in the `ColumnTransformer`, and is it strictly necessary here? It's a safety default — any column not explicitly assigned to a transformer passes through unchanged instead of being silently dropped. In this project all 9 raw feature columns were explicitly covered, so it mainly guards against future schema changes (e.g., someone adding a new raw column).

Q104. What's the difference between imputing with `SimpleImputer(strategy="most_frequent")` vs `strategy="median"` vs `strategy="constant"`? Where did you use each and why? `most_frequent`

(mode) — used for categorical/count-like columns (airline, total_stops). `median` — used for skewed numeric data (duration), robust to outliers. `constant` (fill_value="unknown") — used for additional_info, explicitly marking "missingness" as its own category rather than guessing a value.

Q105. Why is `log1p` used instead of plain `log` for the target transform? `log1p(x) = log(1+x)` safely handles $x=0$ (plain `log(0)` is undefined/-infinity). Even though price is unlikely to be exactly 0 here, it's a standard, safe convention, and its exact inverse `expm1` is used symmetrically at inference.

Q106. What effect does training on `log1p(price)` have on the loss function's behavior? It compresses the scale of large price values, so the model's squared-error loss doesn't get dominated by a few very expensive flights — effectively making the model optimize something closer to *relative* (percentage) error rather than *absolute* rupee error.

Q107. Do you need to inverse-transform predictions before computing R^2 — why? Yes — R^2 computed in log-space and R^2 computed in original-price-space are not the same number and aren't directly comparable to a business audience (who cares about ₹ error, not log-₹ error). You must `expm1()` the predictions back to real price before scoring.

Q108. What's a potential downside of log-transforming the target? The model's errors are now optimized in log-space, which under-penalizes large absolute errors on already-expensive flights (proportionally they look small) — if your business use case actually cares about absolute rupee accuracy on expensive tickets specifically, this could be a mismatch worth reconsidering.

Q109. Why fit the `MeanEncoder/PowerTransformer/etc.` ONLY on training data, never on val/test? To prevent data leakage — if you fit statistics (means, scaling parameters, percentile anchors) using val/test data, information from those "unseen" sets leaks into training, making your evaluation metrics overly optimistic and unrepresentative of true generalization.

Q110. In `app.py`, why is `.transform()` called (not `.fit_transform()`) on the new user input row? Because the encoders/scalers/selector were already fit on `train.csv` when the app started — a new prediction request should only be *transformed* using those already-learned statistics, never re-fit (re-fitting on a single new row would be meaningless and would also constitute leakage/inconsistency).

Q111. What's the difference between ordinal encoding and label encoding, and could using `OrdinalEncoder` incorrectly hurt a model? They're often used interchangeably in practice, but the key issue is that assigning arbitrary integers (0,1,2...) to a *nominal* (unordered) category implies a false numeric order/distance that a model (especially linear ones) might wrongly learn from — that's why `OrdinalEncoder` was used deliberately only for `duration_cat` (which has a genuine order: short<medium<long), not for nominal categories like airline.

Q112. Explain what "curse of dimensionality" means, and how `RareLabelEncoder` helps mitigate it here. As you add more features/columns (e.g., one-hot columns for every rare category), the feature space grows and data becomes sparser relative to that space, making it harder for models to find reliable patterns and easier to overfit. Grouping rare categories into "Other" keeps dimensionality in check.

Q113. What does `SimpleImputer(strategy="constant", fill_value="unknown")` accomplish for `additional_info`, and why not just use most-frequent there too? It treats "missing information" as its own explicit category rather than assuming it's the same as the most common value ("No Info") — potentially useful if missingness itself carries signal (though in this dataset there was no actual missing `additional_info`, this is defensive coding).

Q114. Why did you scale features at all, given `XGBoost` (a tree-based model) doesn't strictly require feature scaling? Mostly for pipeline consistency/reusability and best-practice hygiene — trees split on thresholds and are scale-invariant, so scaling doesn't hurt but also isn't essential for `XGBoost` specifically. It would matter more if you swapped in a linear or distance-based model later.

Q115. What's the difference between a `Pipeline` and a bare sequence of transform calls? A `Pipeline` bundles preprocessing + (optionally) a model into a single object with a consistent `.fit().transform().predict()` interface, ensures steps are applied in the correct order, prevents leakage (fit only happens on `.fit()`, not accidentally on new data), and can be serialized/reused as one unit.

Q116. What would happen if you accidentally called `.fit_transform()` on the test set instead of `.transform()`? The test set's own statistics (means, scales, category groupings) would leak into the transform, and your test predictions would no longer reflect a truly "unseen data" scenario — evaluation metrics would be misleadingly optimistic.

Q117. Explain how `DatetimeFeatures` (from feature-engine) works. It's a transformer that takes datetime columns and automatically extracts requested components (month, week, day_of_week, day_of_year, hour, minute, etc.) as new numeric columns, replacing the need to write manual `.dt.` accessor code for each feature.

Q118. Why extract `day_of_year` in addition to `month`? `month` is coarse (12 buckets); `day_of_year` gives finer-grained resolution within the travel-date range, letting the model potentially learn shorter-term trends/events that a monthly aggregate would smooth over. It was actually the single strongest feature-selection score among date features (0.229).

Q119. What's a potential issue with using `week` and `day_of_year` as two separate features when they're highly correlated with each other? Redundant/correlated features can dilute a tree model's feature-importance signal across near-duplicate columns and don't add much except in high-value edge cases — feature selection (or manually testing) can help you decide if you truly need both.

Q120. What does `FunctionTransformer` let you do, and why is it used so much in this project (part_of_day, is_direct, is_north, is_peak_hour, is_over, have_info)? `FunctionTransformer` wraps any custom Python function into a sklearn-compatible transformer (with `.fit().transform()`), so you can plug arbitrary business-logic/domain-knowledge feature engineering directly into a `Pipeline/FeatureUnion`, keeping everything reproducible and chainable.

Q121. Why write `RBFPercentileSimilarity` as a custom class inheriting `BaseEstimator`, `TransformerMixin` instead of just a function? Because it needs state learned during `.fit()` (the reference percentile values from training data) that must be reused during `.transform()` on new data — a `FunctionTransformer` alone is stateless per-call, so a proper custom estimator class (with `fit/transform` and internally stored `reference_values_`) is the correct sklearn pattern for this.

Q122. What does `BaseEstimator`, `TransformerMixin` give you for free? `BaseEstimator` gives you `get_params()/set_params()` (useful for hyperparameter search/cloning). `TransformerMixin` gives you a free `.fit_transform()` method built from your `.fit()` and `.transform()` implementations.

Q123. What's the trailing underscore convention in `self.reference_values_`? Sklearn convention: attributes ending in `_` are ones that only exist *after* `.fit()` has been called (learned from data), distinguishing them from constructor parameters set at `__init__` time.

Q124. If you deployed this model and traffic patterns shifted (e.g., a new airline enters the market), what would break, and how would you detect it? New/unseen categories would be handled gracefully by `handle_unknown="ignore"` in one-hot encoders (encoded as all-zero) rather than crashing, but the *mean-encoded* location features and RBF percentile anchors would still reflect the old training distribution — this is "data drift," and you'd detect it via monitoring prediction distributions/input feature distributions over time and periodically retraining.

Q125. What would you add to make this feature engineering pipeline more production-robust? Proper out-of-fold target encoding to reduce leakage, schema/input validation before transforming, versioning the preprocessor artifact alongside the model, unit tests for each custom transformer, and monitoring for feature drift.

Section E — Feature Selection (Q126-140)

Q126. What does `SelectBySingleFeaturePerformance` actually do? For each individual feature, it trains a small standalone model (here, a shallow RandomForest) using *only that one feature* to predict the target, scores it (R^2 here), and keeps only features whose standalone score exceeds a threshold.

Q127. Why use `RandomForestRegressor(n_estimators=10, max_depth=3)` specifically for the feature-selection estimator, instead of, say, a deep forest or linear regression? A small/shallow, fast model is intentional here — the goal is a cheap, quick relevance *screening* pass, not a highly accurate model in its own right. Using something small keeps this step fast even though it's repeated once per feature.

Q128. What's the biggest limitation of this feature selection approach? It's univariate/greedy — it evaluates each feature in isolation and completely ignores feature *interactions*. A feature that's weak alone but valuable in combination with another feature would be incorrectly dropped.

Q129. How would you detect/avoid dropping an interaction-only-useful feature? Use a multivariate method instead/in addition — e.g., train the full model with all features and look at permutation importance or SHAP values, or use recursive feature elimination (RFE) which considers the full feature set jointly rather than one-at-a-time.

Q130. Why was the threshold lowered from 0.1 (baseline) to 0.05 (improved) version? To be less aggressive about pruning — keeping mildly-useful features that were previously discarded, betting that XGBoost's own regularization (subsample, colsample, alpha/lambda, gamma) could handle some added noise better than losing potentially useful weak signal.

Q131. What's the trade-off of keeping more (weaker) features vs. fewer (only strong) features? More features = potentially more signal but more noise, higher risk of overfitting on spurious weak features, and larger/slower models. Fewer, stronger features = simpler, faster, more interpretable models but risk losing real (if subtle) signal.

Q132. Which features had the highest standalone R^2 scores, and what does that suggest? `duration` (0.425), `total_stops` (0.403), `duration_cat` (0.395), `is_direct_flight` (0.379) — these dominate individually, suggesting flight length/complexity (duration + number of stops) is the single biggest price driver, more than airline brand or specific route.

Q133. Which features scored near-zero or negative, and were dropped? Things like `airline_Air India` alone (0.0017), `dep_time_part_of_day` (slightly negative), and most `additional_info` one-hot columns — individually weak predictors, likely because their effect is small or only matters combined with other features.

Q134. Can an R^2 score be negative for a single-feature model, and what does that mean? Yes — R^2 compares your model's error to a trivial "always predict the mean" baseline. A negative R^2 means the single-feature model performs *worse* than just guessing the mean every time, i.e., that feature alone is actively unhelpful (or even misleading) as a sole predictor.

Q135. Why fit the feature selector on `log1p(price)` rather than raw price in the improved pipeline? For end-to-end target-space consistency — since the final XGBoost model is trained on `log1p(price)`, evaluating feature usefulness in that same target space avoids any subtle mismatch between what "useful" means during selection vs. what the model actually optimizes for.

Q136. How is feature selection different from dimensionality reduction techniques like PCA? Feature selection *keeps a subset of original, interpretable* features (you know exactly what `duration` means). PCA (and similar) *creates new combined components* that are linear combinations of all original features — often more compact but harder to interpret directly.

Q137. Why might you skip feature selection entirely for a tree-based model like XGBoost?

Tree-based models have built-in feature importance/split-selection and are relatively robust to irrelevant features (they simply won't split on unhelpful ones often) — so aggressive external feature selection is sometimes less critical than for linear models, though it can still help with training speed and reduce overfitting on high-cardinality noise.

Q138. What would you check to validate that the selected 13 (or so) features are genuinely good, beyond their standalone R^2 scores?

Train the final XGBoost model with vs. without feature selection and compare validation performance directly — the "gold standard" check, since standalone univariate scores are only a proxy for actual usefulness in the full model.

Q139. If two engineered features (e.g., `duration` and `duration_rbf_50`) are highly correlated, does the selector remove the redundancy?

No — `SelectBySingleFeaturePerformance` only screens on individual standalone usefulness vs. a threshold; it has no mechanism to detect or remove redundancy/correlation between selected features. Both could be kept even if largely redundant.

Q140. How would you extend this pipeline to also handle multicollinearity among selected features?

Add a correlation-based pruning step (e.g., drop one of any pair with correlation above some threshold) after the univariate selection, or switch to a wrapper method like RFE / use feature importances from the final trained model to prune iteratively.

Section F — XGBoost & Gradient Boosting (Q141-165)

Q141. What is XGBoost, in one sentence? An optimized, regularized implementation of gradient-boosted decision trees, designed for speed and performance on structured/tabular data.

Q142. What is gradient boosting, conceptually? It builds an ensemble of weak learners (usually shallow decision trees) *sequentially*, where each new tree is trained to predict the *residual errors* (or more precisely, the negative gradient of the loss) of the combined ensemble so far — so each tree "corrects" the mistakes of the previous ones.

Q143. How does boosting differ from bagging (e.g., Random Forest)? Bagging trains many trees *independently and in parallel* on bootstrapped samples, then averages them to reduce variance. Boosting trains trees *sequentially*, each one focused on correcting previous errors, primarily reducing bias (and, with proper regularization, controlling variance too).

Q144. Why choose XGBoost over other algorithms (linear regression, plain Random Forest, neural nets) for this problem? Tabular data with a mix of categorical/numeric engineered features and non-linear relationships (as EDA showed) suits tree-based boosting well; XGBoost specifically adds strong built-in regularization and historically has excellent performance/speed on exactly this kind of dataset size and structure, without needing the large data volumes deep learning typically requires.

Q145. What's the role of the learning rate (`eta`)? It scales down each new tree's contribution to the overall prediction (`eta=0.05` means each tree only "counts" 5% as much as it otherwise would) — smaller values mean the model learns more conservatively/slowly, generally improving generalization at the cost of needing more boosting rounds.

Q146. How does `max_depth` affect the bias-variance tradeoff? Deeper trees (`max_depth=6` here) can capture more complex interactions (lower bias) but risk overfitting to training-data noise (higher variance); shallower trees are more constrained/simpler (higher bias, lower variance).

Q147. How does `min_child_weight` act as regularization? It sets a minimum threshold on the sum of instance weights (roughly, sample count) required in a leaf/child node before a split is allowed — this prevents the tree from creating splits based on very few data points, which would likely be overfitting to noise.

Q148. What does `subsample=0.8` do, and why does it help? Each tree is trained on a random 80% subset of rows (without that specific subset being reused unchanged elsewhere) — this stochastic sampling reduces variance/overfitting, similar in spirit to bagging, by preventing all trees from seeing identical data.

Q149. What does `colsample_bytree=0.8` do? Each tree only considers a random 80% of the *features* when building its splits — this decorrelates trees from each other (so they don't all rely on the same dominant feature) and further reduces overfitting.

Q150. What is `gamma` (minimum split loss), and how does it "prune" trees? A split is only made if it improves the loss function by at least `gamma`. Higher `gamma` means the algorithm is more conservative about adding splits — effectively pruning away splits that only marginally help, which are more likely to be overfitting noise.

Q151. Difference between `alpha` (L1) and `lambda` (L2) regularization in XGBoost? Both penalize large leaf weights to control model complexity. L1 (`alpha`) tends to push some leaf weights to exactly zero (sparsity — effectively "turning off" some splits' contribution), while L2 (`lambda`) shrinks weights smoothly toward zero without necessarily zeroing them out.

Q152. What does `objective="reg:squarederror"` mean, and why is it preferred over the older `reg:linear`? It tells XGBoost to optimize Mean Squared Error for a regression task. `reg:linear` was the older name for essentially the same thing, but it was deprecated/renamed because "linear" was a confusing/misleading label (it doesn't mean the *model* is linear — the trees are still non-linear; only the loss function is the standard squared-error one).

Q153. What loss function is being minimized during training? Mean Squared Error between predicted and actual `log1p(price)` values, at each boosting step approximated via gradient (and Hessian, second-order) information for efficient tree construction.

Q154. What is early stopping, and why set `early_stopping_rounds=20`? Training halts if the validation metric (`validation:rmse`) doesn't improve for 20 consecutive rounds, even if `num_round=200` hasn't been reached — this prevents wasting compute on rounds that no longer help (and often no longer help *generalization*, i.e., prevents overfitting from training too long).

Q155. How does XGBoost use second-order (Hessian) information compared to classic gradient boosting? Classic GBM only uses the first-order gradient of the loss to fit each new tree. XGBoost's objective function is a second-order Taylor expansion, using both gradient *and* Hessian (curvature) — this gives a more precise optimization step and is part of why XGBoost often converges faster/better than plain GBM.

Q156. Why is XGBoost typically faster than traditional gradient boosting implementations? Engineering optimizations: parallelized/vectorized split-finding (across features, not across trees, since trees are still sequential), a cache-aware block structure for data storage, approximate histogram-based split-finding algorithms, and out-of-core computation support for very large data.

Q157. What's the difference between XGBoost, LightGBM, and CatBoost at a high level? XGBoost builds trees level-wise (roughly balanced depth-first-ish) with histogram/exact split options; LightGBM builds trees leaf-wise (grows the single most-promising leaf, often faster & more accurate but more overfit-prone on small data); CatBoost has native, often more effective handling of categorical features (via ordered target statistics) without manual encoding.

Q158. What is a `DMatrix`, and why use it instead of a plain numpy array? XGBoost's own optimized internal data structure for training/inference — it stores data in a memory-efficient, cache-friendly format (and can handle sparse data efficiently), which speeds up both training and prediction versus feeding raw arrays directly to a generic model API.

Q159. Why is `num_round=200` combined with `early_stopping_rounds=20` a reasonable setup? It sets a generous upper bound on how many trees *could* be built, while early stopping acts as an

automatic, data-driven "off switch" — so you don't have to manually guess the exact optimal number of rounds; the model stops itself once validation performance plateaus.

Q160. What are the pitfalls of interpreting XGBoost's built-in feature importance (gain/weight/cover)? "Weight" (how often a feature is used to split) can overstate the importance of high-cardinality features that get split on often but not impactfully; "gain" (average loss improvement from splits on that feature) is usually more meaningful but can still be biased toward features with many possible split points; none of these account for feature interactions the way SHAP values can.

Q161. What is SHAP, and how could you use it here? SHAP (SHapley Additive exPlanations) attributes each individual prediction's deviation from the average prediction to each input feature, based on game-theoretic Shapley values — you could use it to explain *why* the model predicted a specific ₹ price for a specific flight (e.g., "duration contributed +₹2,000, is_direct_flight contributed −₹500...").

Q162. Is XGBoost interpretable "out of the box"? Less so than linear regression's coefficients, but more interpretable than deep neural networks — global feature importance gives a rough picture, and per-prediction tools like SHAP make individual predictions explainable, so with the right tooling it can be made reasonably interpretable.

Q163. Why is a tree-based model often preferred over deep learning for structured/tabular data like this? Deep learning typically needs much larger datasets to outperform trees on tabular problems, tends to require more feature-scaling/architecture tuning, and gradient-boosted trees have consistently won most tabular ML competitions (like on Kaggle) at small-to-medium dataset sizes like this one (~6,700 rows).

Q164. If two features are highly correlated (e.g., duration and duration_rbf_50), does that hurt XGBoost the way it hurts linear regression? Much less so — trees just pick whichever correlated feature gives a slightly better split at each node; it doesn't destabilize coefficient estimates the way multicollinearity does in linear regression. It can, however, "dilute" feature-importance credit across the correlated features, making importance rankings a bit less clean to interpret.

Q165. What's the difference between using SageMaker's "built-in" XGBoost container vs. writing your own custom training script? The built-in container just needs data + hyperparameters — AWS manages the training loop/model-saving internally, which is what's used here. A custom "bring your own script" (BYOS) approach gives you full control (e.g., custom preprocessing *inside* the training job, custom logging/callbacks) at the cost of more setup work.

Section G — Hyperparameter Tuning & Bayesian Optimization (Q166-177)

Q166. Why is hyperparameter tuning needed instead of just using reasonable default values? Default hyperparameters are generic guesses; the *optimal* values genuinely depend on your specific dataset's size, noise level, and feature structure — tuning searches for the combination that actually minimizes validation error for *this* problem.

Q167. What is Grid Search, and why wasn't it used here? Grid Search exhaustively tries every combination of hyperparameter values from a predefined discrete grid. It wasn't used because it scales combinatorially (very expensive with 8 hyperparameters) and wastes effort evaluating clearly-bad regions of the search space it hasn't learned to avoid.

Q168. What is Random Search, and how does it differ from Grid Search? Random Search samples random combinations from the specified ranges (rather than an exhaustive grid) — often more efficient than Grid Search because it doesn't waste evaluations on redundant grid points, but it

still doesn't *learn* from past results.

Q169. What is Bayesian Optimization, and how does it improve on Grid/Random Search? It builds an internal probabilistic model (commonly a Gaussian Process) of how the objective (validation RMSE) relates to hyperparameters, based on all past trials — then intelligently picks the next hyperparameter combination that best balances trying promising regions (exploitation) vs. uncertain/unexplored regions (exploration), converging to good solutions in far fewer trials than blind search.

Q170. What's the exploration-exploitation tradeoff in this context? Exploitation = trying hyperparameter values near ones that already worked well. Exploration = trying values in unexplored regions that might be even better but are riskier/uncertain. A good Bayesian optimizer balances both rather than greedily exploiting early good results.

Q171. Why `max_jobs=20` and `max_parallel_jobs=2` instead of higher parallelism (e.g., 20 fully parallel)? Bayesian optimization is inherently *sequential* — each new suggestion benefits from knowing the results of previous jobs. Running many jobs fully in parallel means several jobs get launched *before* earlier results are available to inform them, degrading the "smart" sequential advantage of Bayesian search (though it does speed up wall-clock time).

Q172. Why was `validation:rmse` chosen as the tuning objective rather than R^2 ? RMSE is in the model's native training-loss units (squared error) and is what SageMaker's built-in XGBoost natively reports/optimizes per round — using it directly for tuning is simpler and more standard than computing R^2 for every candidate model.

Q173. What's the mathematical relationship between RMSE and R^2 ? $R^2 = 1 - (SS_{\text{residual}} / SS_{\text{total}})$, where SS_{residual} is essentially $n \times \text{MSE}$ (and $\text{MSE} = \text{RMSE}^2$). So lower RMSE (for a fixed target variance) directly corresponds to higher R^2 — they're monotonically related for a given dataset, just expressed on different scales (RMSE in target units, R^2 unitless 0–1-ish).

Q174. Why `objective_type="Minimize"` for RMSE (rather than "Maximize")? RMSE is an error metric — lower is better — so the tuner is explicitly told to search for hyperparameters that *minimize* it (as opposed to a metric like R^2 or accuracy, where you'd maximize).

Q175. Why were the specific search ranges chosen (e.g., `eta 0.01-0.2`, `max_depth 4-8`)? These are well-established "sane" ranges from general XGBoost tuning best practices/experience — narrow enough to avoid wasting trials on clearly bad extremes (e.g., `eta=1.0` would likely be far too aggressive), wide enough to let Bayesian search actually explore meaningfully within a reasonable region.

Q176. If the best `max_depth` found was exactly the upper boundary (8) of your search range, what would you do next? That's a signal the optimum might lie *outside* the current search range — you'd widen the upper bound (e.g., to 10) and re-run tuning to check if performance keeps improving, rather than assuming 8 is truly optimal.

Q177. What's the risk of tuning hyperparameters using the same validation set that also determines your "final" reported metric? The chosen hyperparameters are implicitly optimized to perform well on that specific validation set, so the validation score can be a slightly optimistic estimate of true generalization — this is exactly why a completely separate, untouched *test* set (evaluated only once, at the very end) is critical, as was done here (Test $R^2=0.79$ nearly matching Val $R^2=0.79$ confirms this wasn't a major issue).

Section H — Model Evaluation & Metrics (Q178-189)

Q178. What is R^2 (coefficient of determination), and how do you interpret $R^2=0.79$? It measures the proportion of variance in the target that's explained by the model, relative to a naive "always predict the mean" baseline. $R^2=0.79$ means the model explains about 79% of the variance in flight prices — a reasonably strong result for a real-world pricing problem with inherent noise.

Q179. Can R^2 be negative? What would that mean? Yes — if the model performs *worse* than simply predicting the mean every time, R^2 goes negative. This happened for some individual weak features during feature selection (e.g., `dep_time_part_of_day`).

Q180. Why report R^2 as the headline metric instead of (or alongside) RMSE/MAE? R^2 is scale-free and easy to communicate ("explains 79% of price variance") to both technical and non-technical audiences, whereas RMSE/MAE are in rupees and require more context (e.g., "is ₹1,500 average error good or bad?") — though reporting both together (as done implicitly via the RMSE-based tuning) gives a fuller picture.

Q181. What is RMSE, and why is it more sensitive to large errors than MAE? Root Mean Squared Error — squares each error before averaging then takes the square root. Squaring disproportionately penalizes large errors (an error of 10 contributes 100 to the sum, not just 10), so a few badly-mispredicted expensive flights would inflate RMSE more than they would MAE.

Q182. What is MAE, and when would you prefer it over RMSE? Mean Absolute Error — average of absolute differences between predicted and actual values. Prefer it when you want a metric that's robust to outliers and directly interpretable as "average rupee error," without letting a few extreme mispredictions dominate the number.

Q183. Train $R^2=0.91$ vs Test $R^2=0.79$ — is this overfitting? How would you diagnose it further? There's a meaningful gap (~ 0.12), suggesting some overfitting despite the regularization in place. To diagnose further: check learning curves (train vs val error as training progresses), try k-fold cross-validation to see if the gap is consistent across folds, and inspect whether specific feature groups (e.g., rare category one-hot columns) are driving memorization.

Q184. What steps would you take to close that train-test gap? Increase regularization (higher `alpha/lambda/gamma`, lower `max_depth`), reduce `num_round` or make early stopping more aggressive, gather more training data if possible, or apply more careful (out-of-fold) target encoding to reduce leakage-driven overfitting in the mean-encoded location features.

Q185. Why is Val R^2 (0.79) being close to Test R^2 (0.79) a good sign? It shows the hyperparameter tuning process, which used the validation set, didn't "overfit to the validation set" itself — if it had, you'd expect test performance to be noticeably worse than validation performance on this held-out, never-touched-during-tuning set.

Q186. What would it mean if Test R^2 had come out much lower than Val R^2 (e.g., 0.79 vs 0.55)? It would suggest the tuning/model-selection process overfit to the validation set specifically (or that the test set's distribution genuinely differs from train/val — worth double-checking the random split for any data leakage or distributional shift).

Q187. Beyond R^2 /RMSE, what business-relevant metric might you report to a non-technical stakeholder? Something like "average absolute prediction error in ₹" (MAE) or "% of predictions within $\pm 10\%$ of actual price" — more intuitive for a business audience than a statistical R^2 number.

Q188. How would you check if the model's errors are systematically biased for specific airlines or routes, rather than just looking at one aggregate R^2 ? Compute residuals (actual – predicted) grouped by airline/route/stop-count and look for systematic over/under-prediction patterns (e.g., via a bar chart of mean residual per airline) — a single overall R^2 can hide meaningfully poor performance on specific important subgroups.

Q189. If asked "how confident are you in this 0.79 R^2 ", how would you convey uncertainty around it? Report it with a confidence interval (e.g., via bootstrapping — resampling the test set

repeatedly and recomputing R^2 each time to see the spread) rather than a single point estimate, and ideally cross-validate across multiple different train/test splits to check stability.

Section I — AWS SageMaker & Cloud Training (Q190-201)

Q190. What is AWS SageMaker, in one sentence? A fully-managed AWS service that handles the infrastructure (compute provisioning, containerization, scaling) for building, training, tuning, and deploying machine learning models, so you don't have to manage servers yourself.

Q191. Difference between using a SageMaker "built-in algorithm" container vs. bringing your own training script? Built-in (used here) needs only your data + hyperparameters — AWS's pre-packaged Docker image handles the training loop and model-saving internally. "Bring Your Own Script/Container" gives full control over the training code (e.g., custom preprocessing inside the job, custom logging) at the cost of more setup/maintenance work.

Q192. Why upload preprocessed data to S3 before training, rather than passing it directly? SageMaker training jobs run in isolated, ephemeral compute instances that read their input data from S3 (not from your local notebook's memory/filesystem) — S3 is the standard, scalable, durable storage layer that decouples data from compute in the AWS architecture.

Q193. What format restrictions does SageMaker's built-in XGBoost require for CSV input, and why? No header row, and the target column must be the *first* column, followed by feature columns — this is simply the fixed convention the built-in algorithm's internal data-reading code expects.

Q194. What is a "training channel," and what are the "train"/"validation" channels used for here? A named S3 data source SageMaker feeds into the training container. The "train" channel is used to fit the model; the "validation" channel is used during training to compute `validation:rmse` after each round (for early stopping and for reporting the tuning objective).

Q195. What is a Spot Instance, and why use one here? Spare/unused AWS EC2 compute capacity offered at a steep discount (often 70-90% off on-demand pricing) — used here (`use_spot_instances=True`) purely for cost savings during training, since 20 tuning jobs can add up in compute cost.

Q196. What's the risk of Spot Instances, and how did the code account for it? AWS can reclaim/interrupt a Spot Instance with little notice if it needs the capacity back. `max_wait=1200` (vs `max_run=600`) gives SageMaker extra buffer time to retry/resume the job if interrupted, before ultimately giving up.

Q197. Why `ml.m5.xlarge` and not a GPU instance for this task? XGBoost training on a modest tabular dataset (~6,700 rows, ~13-31 features) is CPU-efficient and doesn't benefit much from GPU acceleration at this scale — a general-purpose CPU instance is both sufficient and more cost-effective than a GPU instance here.

Q198. What is an IAM execution role, and why does SageMaker need one? An AWS Identity and Access Management role that grants the SageMaker training job permission to access other AWS resources it needs (like reading/writing to your specific S3 bucket) — without it, the training job wouldn't be authorized to read your uploaded data or save its output model.

Q199. What does `model.tar.gz` contain, and why package models this way? A compressed archive containing the trained model artifact (here, the pickled XGBoost Booster). Packaging as a single tarball is SageMaker's standard convention for storing/transferring trained-model outputs to/from S3 consistently across all algorithm types.

Q200. How does SageMaker's `HyperparameterTuner` determine which of the 20 jobs was "best"? It tracks the `objective_metric_name` (`validation:rmse`) reported by each of the 20 training jobs and, since `objective_type="Minimize"`, simply selects the job with the lowest reported validation RMSE as the best.

Q201. What cost considerations should you think about before launching 20 Bayesian tuning jobs? Total cost $\approx$ (per-job training time) $\times$ (number of jobs) $\times$ (instance hourly rate), reduced by the Spot Instance discount — worth estimating upfront, and worth setting `max_jobs/max_parallel_jobs` and instance size deliberately (as done: modest `ml.m5.xlarge`, and capped at 20 jobs) rather than open-ended tuning that could run up unexpected costs.

Section J — Streamlit App, Deployment & MLOps (Q202-213)

Q202. Why does `app.py` re-fit the entire preprocessor at startup instead of loading a saved `preprocessor.pkl`? To avoid pickle's version-compatibility fragility — a pickled object containing custom classes (like `RBFPPercentileSimilarity`) can fail to load if the deployment environment's Python/sklearn/feature-engine versions differ even slightly from the training environment's. Refitting fresh on `train.csv` guarantees compatibility with whatever versions are actually installed on Streamlit Cloud.

Q203. What's a downside of this "refit on every app startup" approach? Slower app startup (recomputing all encoders/scalers/percentiles from scratch each time), duplicated code between the training notebook and `app.py` (risk of the two drifting out of sync if one is updated without the other), and no caching across sessions.

Q204. Why convert the XGBoost model itself to JSON format instead of also refitting/retraining it live? The model's learned weights (from expensive SageMaker Bayesian-tuned training) shouldn't be recomputed live — that would be wasteful and non-reproducible session-to-session. JSON format specifically solves XGBoost's own version-portability problem without needing to retrain.

Q205. What is `joblib`, and why might you use it instead of plain `pickle` for the sklearn preprocessor? `joblib` is optimized for efficiently serializing objects containing large numpy arrays (common in fitted sklearn transformers) — generally faster and more memory-efficient than plain `pickle` for such objects, though it shares the same underlying version-fragility issue that motivated the "refit fresh" design here.

Q206. Why does `app.py` cast the date/time Streamlit widget outputs to strings before passing them into the pipeline? Streamlit's `date_input/time_input` return native Python `date/time` objects, but the trained pipeline's transformers (`pd.to_datetime`, `DatetimeFeatures`) expect string-parseable input matching the format seen during training on the CSV data (where dates/times were originally strings).

Q207. What's the purpose of the `try/except` around the prediction logic? To catch any runtime errors gracefully (e.g., an edge-case input the pipeline doesn't handle well) and show a user-friendly error message via `st.error()`, rather than letting the whole Streamlit app crash.

Q208. If you wanted to avoid re-fitting the preprocessor on every app run, what would you use? Streamlit's `@st.cache_resource` decorator, which caches the fitted preprocessor object across reruns/sessions within the app's lifetime, so the expensive fit only happens once instead of on every user interaction/rerun.

Q209. How would you add basic production hardening (auth, rate limiting) to this app if needed? Put it behind an authentication layer (e.g., Streamlit's built-in auth options, or a reverse proxy with API keys), add request throttling/rate limiting at the infrastructure level, and validate/sanitize all user inputs before they reach the model pipeline.

Q210. What would you monitor in production for this deployed model, beyond simple uptime? Input feature distributions (to detect data drift vs. training data), prediction distribution over time (sudden shifts could signal a problem), latency, error rates, and periodically re-evaluating accuracy against any new ground-truth prices that become available.

Q211. How would you set up automatic retraining if flight pricing patterns shift over time (concept drift)? Schedule periodic retraining (e.g., monthly) on freshly collected data, monitor a live validation metric against new labeled data to trigger retraining when performance degrades past a threshold, and version each model/preprocessor pair so you can roll back if a new version underperforms.

Q212. What testing would you add before deploying an updated model to replace the current one? Shadow testing (run the new model alongside the old one on live traffic without serving its predictions, comparing outputs) and/or A/B testing (serve a small % of traffic the new model and compare business metrics) before fully cutting over.

Q213. Why might it matter that `requirements.txt` doesn't pin exact library versions (e.g., just `xgboost` not `xgboost==1.7.0`)? Unpinned versions mean a future `pip install` could pull a newer library version with breaking changes (e.g., different default parameters, deprecated APIs) — risking silent behavior changes or crashes; pinning exact versions makes deployments reproducible and predictable.

Section K — Project-Level, System Design & Behavioral (Q214-221)

Q214. Walk me through this project end-to-end in about 2 minutes. "I built an Indian flight-price predictor: cleaned a raw 10.6k-row dataset (fixing types, duplicates, a bad row), split it into train/val/test, ran deep EDA with hypothesis testing (ANOVA, correlation tests, normality tests) to understand what actually drives price, engineered ~30 features via a modular sklearn pipeline (encoding, RBF-kernel duration similarity, domain flags like peak-hour/is-north), selected the most useful ~13-15 via univariate feature selection, then trained an XGBoost model on AWS SageMaker with Bayesian hyperparameter tuning across 20 jobs — landing at $R^2=0.79$ on a held-out test set — and deployed it as a live Streamlit app."

Q215. What was the single most impactful feature-engineering decision, and why? Engineering `duration` and `total_stops` deeply (RBF similarity, categorical bucketing, direct-flight flag) — feature selection showed these had by far the highest standalone predictive power (R^2 up to 0.43), confirming flight length/complexity is the dominant price driver in this dataset.

Q216. What's the weakest part of this pipeline, and why? The mean/target encoding for `source/destination` has real (if modest, given train-only fitting) leakage risk since it's not done with proper out-of-fold cross-validation — and the univariate feature selector ignores feature interactions, potentially discarding useful combined signals.

Q217. Walk through the 5 differences between the "baseline" (`FeatureEngineering.ipynb`) and "improved" (`TrainingData.ipynb`) pipelines. (1) Added `is_north` location flag, (2) added `is_peak_hour` time flag, (3) expanded RBF anchors from 3 to 5 percentiles, (4) lowered feature-selection threshold from 0.1 to 0.05, (5) switched to training on `log1p(price)` instead of raw

price.

Q218. If your manager asked "can we trust this model's predictions for real pricing decisions" given $R^2=0.79$, what would you say? It explains a strong majority of price variance and is a solid *decision-support signal* (e.g., flagging if a listed price looks unusually high/low), but I wouldn't fully automate pricing decisions on it alone — the ~21% unexplained variance likely includes real-world factors not in this dataset (demand surges, seat availability, promotions), so I'd pair it with human review or additional live signals for high-stakes decisions.

Q219. What business assumptions does this model implicitly make? That the pricing patterns present in the (2019-era) training data still hold — airline pricing strategies, fuel costs, and demand patterns can shift significantly over time, so the model would need retraining to stay reliable for current predictions.

Q220. How would you explain "data leakage" to a non-technical stakeholder, using an example from this project? "It's like grading a student's exam using answers they already saw beforehand — the model would look like it's performing great during testing, but that's an illusion, because in the real world it won't have access to that same 'peek.' In our project, this could happen if we let the price-based encoding for a route 'see' that exact flight's own price when learning the route's typical price."

Q221. What's one thing you'd do differently about the train/val/test split strategy, given flight prices can have real temporal trends? Consider a time-based split (train on earlier dates, test on later dates) instead of a purely random split — a random split can let the model "see" patterns from dates that are chronologically *after* some of its validation/test dates, which doesn't reflect the real-world scenario of predicting *future* prices from *past* data.

Quick Revision Cheat-Sheet

Topic	One-line Answer
Dataset	10,683 → 10,462 rows after cleaning (220 dupes + 1 bad row removed)
Split	Train 6,695 / Val 1,674 / Test 2,093 (≈64/16/20%)
Missing values	Only 1 (<code>total_stops</code>), filled with 0
Normality tests used	Shapiro-Wilk, Anderson-Darling
Correlation tests used	Pearson, Spearman
Categorical association tests	Chi-square, Cramér's V
Group-difference tests	ANOVA, Kruskal-Wallis
Duration feature engineering	Winsorizer → RBF kernel similarity (5 percentile anchors) + category bucket + over-1000 flag + StandardScaler

Location encoding	RareLabel → MeanEncoder → PowerTransformer + <code>is_north</code> flag
Feature selection method	<code>SelectBySingleFeaturePerformance</code> (RandomForest, R^2 , threshold=0.05)
Target transform	<code>log1p(price)</code> , inverted with <code>expm1()</code> at prediction time
Model	XGBoost 1.2-1, <code>reg:squarederror</code> , 200 rounds, early stopping @ 20
Tuning	AWS SageMaker Bayesian HPO, 20 jobs, 2 parallel, minimize <code>validation:rmse</code>
Final performance	Train $R^2=0.91$, Val $R^2=0.79$, Test $R^2=0.79$
Model format for deployment	JSON (<code>xgboost-model.json</code>) — version-independent
Deployment	Streamlit app; preprocessor refit fresh at startup (avoids pickle version issues)

Good luck bhai! Poore project ka flow, har statistical test ka "why", aur har feature-engineering choice ka reasoning — sab yahan hai. Interview se pehle "Quick Revision Cheat-Sheet" ek baar zaroor daudha lena.